



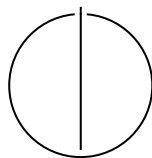
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Design and Performance Analysis of a
Deterministic Network Stack for Virtualised
Protection and Control Systems**

Radu-Vladimir Raceala





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Design and Performance Analysis of a Deterministic
Network Stack for Virtualised Protection and Control
Systems**

**Entwurf und Leistungsanalyse eines
Deterministischen Netzwerkstacks für Virtualisierte
Schutz- und Steuerungssysteme**

Author:	Radu-Vladimir Raceala
Examiner:	Prof. Dr. Pramod Bhatotia
Supervisors:	Peter Okelmann, M.Sc., Dr. Anatole Lefort
Submission Date:	06.09.2026

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 06.09.2026

Radu-Vladimir Raceala

Acknowledgments

I would like to thank my advisors, Peter Okelmann and Anatole Lefort, for their guidance throughout this thesis. Anatole Lefort set the initial direction of the work, and its completion would not have been possible without Peter Okelmann, who advised me both during our weekly meetings and outside of them. I am also grateful to Prof. Pramod Bhatotia and the Systems Research group for providing an environment where I could learn deeply about the topics that interest me.

Abstract

The protection infrastructure of electrical grids is increasingly put under pressure by the variability in energy sources. Hardware devices are tasked with measuring voltage and current, and with processing the resulting values to detect faults. Virtualisation lowers costs and increases flexibility by enabling protection functionality to run on a central server. The network stack has the critical function of delivering current and voltage measurements to the Virtualised Protection and Control (VPAC) systems, and of transmitting trip commands when faults are detected. It must provide isolation and latency guarantees without compromising protection and control functionality, and it must integrate with the communication protocols defined in the IEC 61850 standard. The fault-detection cost is small and has low variance, so the latency outliers must stem from the inefficient delivery of sensor values. To reduce the magnitude and frequency of such occurrences, special attention must be directed at the network stack.

We analyse the viability of implementing redundancy and forwarding in software for protection workloads. We present three network stacks and discuss their tradeoffs with respect to end-to-end latency, determinism, interoperability and resource usage. We investigate the efficacy of kernel bypassing techniques in reducing protection loop latencies, comparing their effects across substation sizes. For that purpose, we design an Inter-Process Communication protocol for efficient data flow in the virtual machines, and a deduplication algorithm for integration with the Parallel Redundancy Protocol. We iteratively model system capacity and latency, and validate the models against measurements. Our evaluation shows that VPAC systems do not need specialised hardware to fulfil their tasks. The source code can be found at <https://github.com/radurac/vpac-bsc-thesis-artifact>.

Contents

Acknowledgments	iii
Abstract	iv
1 Introduction	1
2 Background	4
2.1 IEC 61850	4
2.2 Centralised Protection and Control	5
2.3 Parallel Redundancy Protocol	6
2.4 Time-Sensitive Networking	9
2.5 Linux Networking Stack	9
2.5.1 Interrupt Requests and Software Interrupts	9
2.5.2 Linux Bridge	10
2.5.3 Receive Side Scaling	10
2.6 Kernel Bypassing	10
2.6.1 Data Plane Development Kit	11
2.6.2 eXpress Data Path	11
2.7 Generic Architecture	12
2.7.1 Substation Grid	12
2.7.2 Link Redundancy Entity	13
2.7.3 Forwarding Unit and Virtual Devices	13
2.7.4 VPAC	13
3 Overview	15
3.1 Shared-Nothing	15
3.1.1 Workflow	15
3.2 AF_XDP-based Host-Cache	17
3.2.1 Workflow	17
3.3 DPDK-based Host-Cache	18
3.3.1 Workflow	18
3.4 Design Goals	18

4	Design	21
4.1	VM Resource Assignment	21
4.2	Shared-Nothing Variant Components	22
4.2.1	PRP Kernel Module	22
4.2.2	L2 Forwarding for SV	23
4.2.3	VPAC-to-VM Assignment	23
4.3	Components of the Host-Cache Variants	25
4.3.1	Modified VPAC	25
4.3.2	Host-Cache	26
4.3.3	Deduplication Algorithm	27
4.4	Hardware Alternatives	28
5	Implementation	30
5.1	Shared-Nothing Variant Implementation	30
5.2	Implementation of the Host-Cache Variants	31
5.2.1	eBPF Implementation of PRP	31
5.2.2	Host-Cache Implementation	34
5.2.3	Eventfd IPC Protocol	37
5.3	Watchdog	39
5.4	Handling Best-Effort Traffic	39
6	Evaluation	41
6.1	Models	41
6.1.1	Overview and Motivation	41
6.1.2	Assumptions	43
6.1.3	Capacity Models	43
6.1.4	DPDK Variant Worst-Case Burst Model	46
6.1.5	Estimation of Constant Costs	47
6.2	Measurement Setup	48
6.2.1	Load Generator	49
6.2.2	Test Bench	49
6.2.3	Configuration	49
6.3	Host-side CPU Usage	51
6.4	Round-Trip Time	53
6.5	Model Validation	55
6.5.1	Host-Cache VM Capacity	55
6.5.2	Worst-Case Burst	56

7	Related Work	60
7.1	Containerised VPACs	60
7.2	Industry White Papers	60
7.3	DPDK-based Low-Latency Applications	61
7.4	Kernel Bypassing for TSN	62
8	Summary and Conclusion	64
9	Future Work	66
	Abbreviations	68
	List of Figures	70
	List of Tables	72
	Bibliography	73

1 Introduction

Although Ethernet communication via the IEC 61850 standard has brought substation grids unprecedented interoperability and standardisation at the level of the protection and control layer since its worldwide adoption in the early 2000s [53], the rise of renewables has shown that changes have to be made for the circuit protection subsystem to remain reliable. Under the new conditions, feeders are connected and disconnected far more frequently. Even a medium-voltage substation requires up to 100 vendor-specific Intelligent Electronic Devices (IEDs) [36], which slow down the extension of a system that must stay flexible. Centralised Protection and Control (CPC) is the natural successor to conventional IED-based protection: instead of configuring and connecting many hardware devices that already communicate using Ethernet, there is a central server that implements the protection and control functionality in software and that integrates into the substation grid using the same IEC 61850 standard [34].

However, one negative aspect remains to be mitigated, namely the problem of isolation. Many other non-essential services run on the central server, and these must not violate the Quality of Service (QoS) guarantees that are enforced by the standard. The solution is virtualisation: the CPC systems run in isolated Virtual Machines (VMs), which we call Virtualised Protection and Control (VPAC) systems, each pinned to a different physical core.

Protection algorithms exhibit predictable memory and compute consumption. They do not demand many resources. Therefore, the delivery system of the current and voltage values is of interest for lowering the reaction time of the VPAC, rather than the protection mechanism itself. Adjustments have to be made to the networking stack that services VPACs so that the latency bounds are met.

There already exist ongoing projects and finished products that implement centralised protection and control through virtualisation [2, 75, 10]. Although they are robust, industry-tested solutions, two problems arise from a research perspective. First, they generally assume or make use of specialised hardware, like Single Root I/O Virtualization (SR-IOV) or redundancy-aware Network Interface Cards (NICs). Often, software implementations of the relevant protocols are considered inefficient alternatives. Second, to the best of our knowledge, there is no comprehensive comparison between different network stacks from within the same evaluation framework. No attempt has been made to model and decompose latencies, or to model the capacity of the system,

that is, to empirically or theoretically determine the maximum substation size so that the latency bounds still hold for a given resource pool. Modularity, flexibility and scalability have a different meaning in a constrained environment like that of hard real-time protection and control, and a system cannot have those properties in such a context without adequate worst-case analysis.

To make a centralised and virtualised solution as attractive as possible to the industry, we analyse whether NICs with common capabilities paired with software implementations of redundancy and switching can comply with QoS guarantees enforced by the IEC 61850 standard. Because the Linux kernel is too complex for a complete worst-case execution time analysis, we approach the problem of determinism empirically, while iteratively developing more accurate models of the latency and of the system capacity. Besides pure performance, predictability is a constant factor in navigating the design space. Considering the tradeoffs with respect to the different goals, we design three network stacks: a Shared-Nothing variant based on components provided by the standard Linux stack, and two Host-Cache variants that store processed sensor values and leverage kernel bypassing for determinism and performance, while demanding VPAC modification. One of the Host-Cache variants is based on the eXpress Data Path (XDP), and it represents a partial version of kernel bypassing. The other one swaps the standard interrupt-driven network driver for a Data Plane Development Kit (DPDK) Poll Mode Driver (PMD) to eliminate variance in the early stages of packet processing.

We evaluate the layers of the stack independently and we conclude that DPDK-based kernel bypassing allows for larger substations to be serviced with similar or lower resource usage, while offering higher predictability. In particular, the solution allows for a latency decomposition model that accurately describes system behaviour when load increases. Our measurements also show that software implementations of Parallel Redundancy Protocol (PRP) remain viable, and that most time is lost either in the virtual network or in the management of the VPACs in the virtual machine layer. As such, our main contributions are the following:

1. We design three network stacks that integrate with the communication protocols defined in the IEC 61850 standard and with PRP, and we compare them from within the same evaluation framework, taking into consideration end-to-end latency, determinism, interoperability and resource usage.
2. We analyse whether software implementations of PRP abide by the necessary QoS guarantees. We propose a deduplication algorithm for PRP and compare it to already available implementations.
3. We develop an Inter-Process Communication (IPC) protocol in the VM layer to

mitigate busy polling costs.

4. We model the capacities of the VPACs, and develop a latency decomposition model for the DPDK-based variant.

The remainder of the thesis opens in Chapter 2 with a presentation of the background information, centered on the IEC 61850 standard, related standards and technologies, and technical information about the Linux kernel networking stack and its limitations. Chapter 3 provides a brief description of the designs and workflows of the three proposed solutions, while also setting out the design goals. Chapter 4 presents each individual component in detail, marking the main differences between software and hardware solutions for deduplication and forwarding. At a lower level of abstraction, Chapter 5 discusses the concrete implementations and best-effort traffic handling. The modelling framework is described in Chapter 6, together with benchmarks that either validate the models, or provide information that cannot be extracted theoretically. The thesis ends with a survey of related work (Chapter 7), a conclusion regarding the findings (Chapter 8), and a description of unexplored directions (Chapter 9).

2 Background

This chapter introduces the substation-specific technical standards that may be unfamiliar to the reader. It presents relevant parts of the Linux networking stack, centered around interrupt handling. At the end of the chapter, we discuss kernel bypassing as a method to avoid certain limitations of the native stack, and, to prepare later developments, we describe the virtualised and software-based solution in its most generic form.

2.1 IEC 61850

IEC 61850 is the standard for Ethernet-based communication in substations, which are essential power grid components [52]. IEC 61850 defines three levels of communication: the process level, directly connected with sensing and breaking devices; the bay level, which contains IEDs that handle protection and control; and the station level, which is reserved for configuration and human intervention. The station bus and the process bus connect the three levels.

Generic Object-Oriented Substation Events (GOOSE) is the protocol used for irregular emergency communication (e.g., in case of overcurrent) and has latency requirements between 4 ms and 1 s. Sampled Values (SV) is the constant-bandwidth transport protocol for digitised current and voltage values, which are packaged and transformed by the Merging Unit (MU) before transmission. For example, for a signal frequency of 50 Hz and 80 samples per cycle, the sending time interval of the MU is 250 μ s. These values must reach the IEDs, which extract the samples and examine them to detect faults. If faults are discovered, the IEDs issue a trip message encapsulated as a GOOSE packet to the Circuit Breaker (CB), which then trips the switchgear [45]. Both of these are Publisher-Subscriber protocols based on multicast MAC addresses. The Ethernet payload of an SV message contains one or more Application Specific Data Units (ASDU), each being composed of three phase current and voltage measurements, and accounting information that includes a stream identification value and a unique per-ASDU identification value [33].

Interoperability and further standardisation are ensured by the XML-based Substation Configuration Language and by the data model. In particular, the data model was

unique at the time IEC 61850 was first released [53]. It provided all devices with a consistent way of storing substation data and it lowered the chance of human errors.

2.2 Centralised Protection and Control

The Centralised Protection and Control (CPC) system provides protection, automation control and asset management functions for a substation, marking a shift from discrete, proprietary IEDs to a central solution [34]. This shift is based on the IEC 61850 communication protocols that allow for decision-making to be moved away from the sensors, but kept within reach of an Ethernet network. Conventionally, IEDs are directly connected to devices such as voltage or current sensors and circuit breakers over copper wires, as shown in Figure 2.1. Instead, CPC systems communicate over Ethernet with MUs that digitise the measurements, aggregating protection functionality and reducing the number of hardware devices needed to service the substation, as in Figure 2.2.

Scalability, defined in terms of unforeseen extension of the substation, is the main respect in which CPC systems outperform conventional ones. Because much of the functionality is moved from the IEDs to software, the number of hardware devices that have to be connected to the substation grid decreases considerably. However, scalability must not interfere with the latency guarantees. Protection is classified as hard real-time; as such, accommodating more feeders than planned is only an option if the limits of the system are well understood. Consequently, this is the main focus of the thesis and often informs design decisions for the network stack.

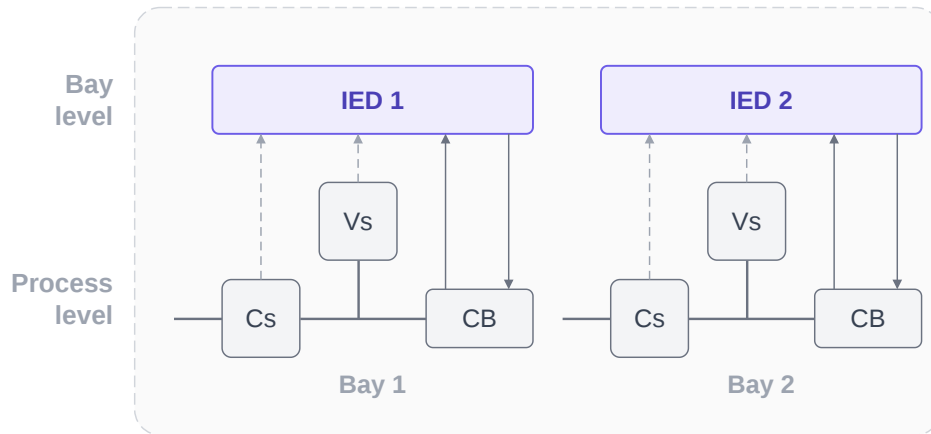


Figure 2.1: The conventional architecture.

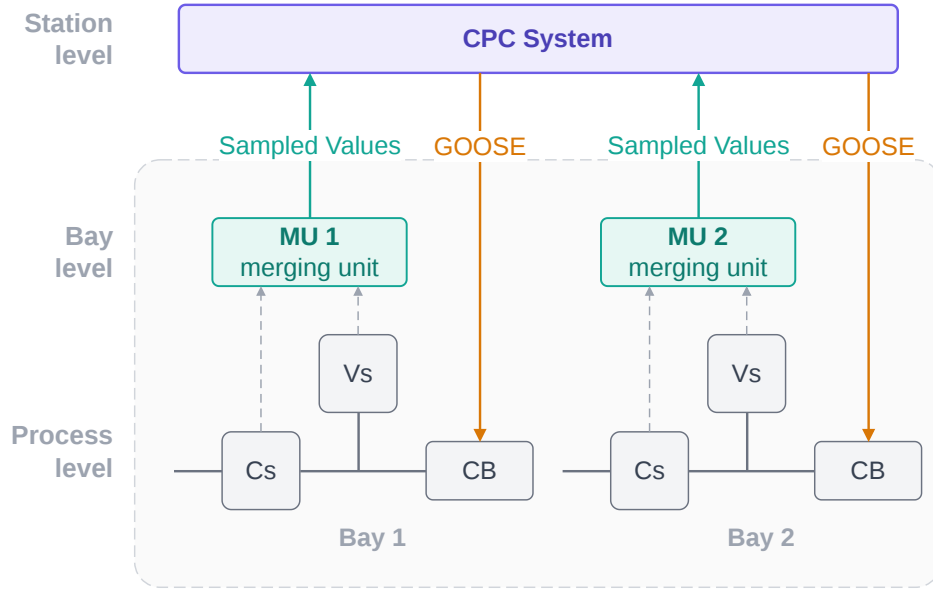


Figure 2.2: The CPC architecture.

2.3 Parallel Redundancy Protocol

Parallel Redundancy Protocol (PRP), as defined in the standard IEC 62439 [35], provides availability in case of single failures in Ethernet networks. A Doubly Attached Node implementing PRP (DANP) is connected to two independent LANs simultaneously, the same information flowing in parallel through both. As such, the DANPs have to duplicate each frame and send it to the destination via both networks, and deduplicate on receive, that is, identify the first arrival while discarding the duplicate. Correct deduplication rests on the information provided in the Redundancy Control Trailer (RCT), which is attached to the end of each frame. It is made of fields that identify the protocol, and a sequence number which, combined with the source MAC, uniquely identifies the frame, as in Figure 2.3. This pair contains all the information necessary to correctly identify first arrivals and duplicates. Singly Attached Nodes (SANs) can be integrated. They can be connected to both LANs if they interface with a Redundancy Box (RedBox) that implements the protocol for them in hardware, or to one single LAN, in which case they can communicate with all DANPs (non-redundantly), but only with the SANs on that same LAN. Figure 2.4 displays an example of a PRP-compliant network topology.

While the standard does not enforce a specific algorithm for deduplication, it men-

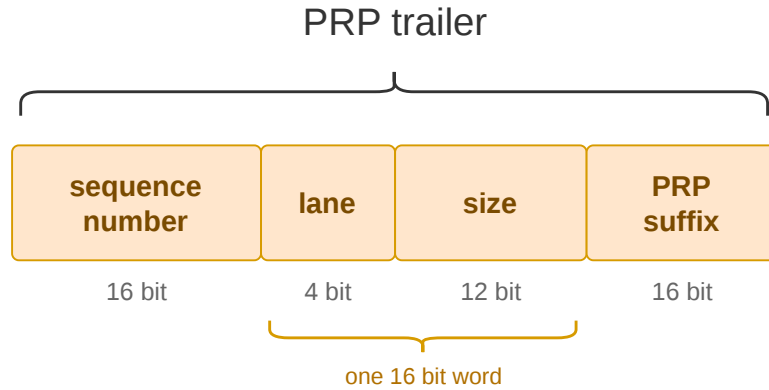


Figure 2.3: The redundancy control trailer as defined in the standard. It is attached at the end of the payload.

tions assumptions that should not be made about the network when implementing the algorithm. The LANs can induce asymmetric delays, although they should have a similar topology. On reboot, the first sequence number is not fixed. The implementation may at times allow both copies to go up the network stack, but it should never incorrectly reject both copies. There is published work that discusses different approaches [6, 49]. A comparison between implementations is the subject of later sections.

For monitoring purposes, each node must count per-LAN errors, as well as the number of received frames. Additionally, a node table containing information about each node can be kept.

The protocol is paired with the IEC 61850 suite to ensure that the sensor values arrive at the CPC unit, and that all devices can be notified of important GOOSE events, in spite of localised hardware failure or potential forwarding errors.

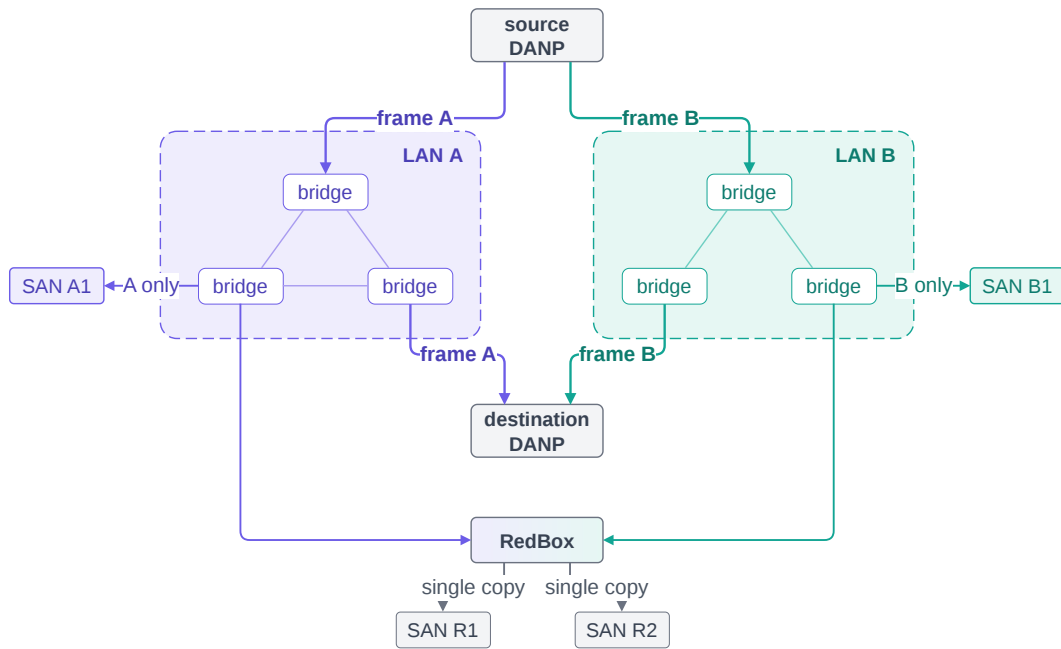


Figure 2.4: Example of a PRP-compliant network topology. DANPs can communicate with all other nodes. SANs can communicate with DANPs, or SANs on the same LAN. A RedBox is a hardware device that allows a SAN to act as a DANP. The precise layout of the LANs does not have to be the same. The figure is a simplification of one in [35].

2.4 Time-Sensitive Networking

Time-Sensitive Networking (TSN) started in the context of audio-video bridging, but has since expanded into industrial automation because of shared goals such as bounded latency and zero packet loss [57]. Managing best-effort and critical traffic at the same time is difficult under the original design of Ethernet; TSN builds on top of it to provide tools that allow for differentiated handling of flows.

Bridges that implement transmission gates are one such tool. A periodic window is defined for each bridge in the network, and during this window only critical traffic can be transmitted. To ensure that no best-effort packet that began serialisation just before the deadline blocks the window until its transmission is complete, another protected section has to be enforced before the window, with its length given by the size of the largest non-preemptible segment. Industrial automation networks need to periodically transport traffic from sensors to controllers and back. For IEC 61850-based substation grids, SV defines the sensor-controller flow, while GOOSE defines the controller-sensor flow. Because of the cyclical nature of the traffic (for instance, SV streams transmitted every 250 μ s), mechanisms such as transmission gates are well suited for QoS guarantees. The requirements for deterministic real-time communication of industrial automation can be met with Class C Real-Time Ethernet solutions implemented over TSN [81]. Such networks can achieve cycle times of less than 1 ms.

2.5 Linux Networking Stack

2.5.1 Interrupt Requests and Software Interrupts

IRQs, or hardware interrupts, represent the notification mechanism by which the hardware signals the completion of an I/O operation [17]. The interrupt usually preempts the currently running code, executes as little as possible and defers the rest of the handling to a so-called softIRQ. This mechanism exists because other hardware interrupts may be disabled during the execution of an interrupt handler, which could cause critical events to be missed [20]. In a networking context, software interrupts fire when a new frame is received (`NET_RX_SOFTIRQ`), and also when the NIC has completed transmitting a frame (`NET_TX_SOFTIRQ`).

In the past, each frame triggered an IRQ, which was problematic at high throughputs, because the CPU spent all of its time in interrupt processing [22]. This issue, called receive livelock, is what New API (NAPI) addresses; it is the software equivalent of hardware interrupt throttling mechanisms. After the hard IRQ is serviced, the NAPI loop is scheduled. This runs either until there are no more frames left in the receive queue to process or until a predefined budget is depleted. The latter mechanism ensures

fairness. During this entire period, the corresponding IRQ (belonging to the receive queue) is masked out [8], which means that it cannot trigger. There are important differences in how a PREEMPT_RT kernel handles interrupts compared to a stock kernel. First, interrupts are force-threaded. Second, the softIRQ runs in thread context, and as such, can be preempted by other threads with higher priority [76]. This is necessary for determinism, but can potentially lower throughput because of the introduced context switches.

2.5.2 Linux Bridge

A Linux bridge is a kernel module that acts as a network switch. It can be used to forward frames between the host and deployed VMs, between the VMs themselves, or in general between virtual network devices [30]. It also supports multicast traffic, either via snooping or via static mappings that can be added to the multicast database. For traffic to flow through the bridge, the targeted physical and virtual interfaces have to be enslaved to it [58]. Each interface can only be enslaved to one bridge at a time.

2.5.3 Receive Side Scaling

Because the softIRQ code generally runs on the same core where the original hardware interrupt fired, being limited to one interrupt line (that is, one receive queue) would entail that the Linux networking stack's throughput is bounded by one core's capacity. However, that is not the case: contemporary NICs support more than just one receive queue [72]. Such NICs compute a hash function over the headers of the different protocols, and then assign all packets that hash to the same value to a certain receive queue via an indirection table. This mechanism is known as Receive Side Scaling (RSS). There exist more advanced dynamic implementations such as RSS++ [7], which can provide a better spread of packets.

By using `ethtool`, the user can set the number of receive queues, and thereby the number of IRQs. While the standard low-latency setup is to have as many receive queues as the number of cores, or the maximum number of receive queues supported by the NIC if the number is lower than the core count, there are scenarios in which this is not optimal. This is a byproduct of NAPI.

2.6 Kernel Bypassing

The kernel networking stack was built to be flexible and to provide good performance on average for generic workloads. Many different kernel mechanisms are involved in packet processing, as seen in the previous sections, and user-space applications have

no control over them. Kernel bypass techniques move control of networking into the user-space application, to avoid kernel-user-space context switches, unnecessary copy operations, and data structure allocations [31, 9]. The DPDK framework and the XDP hook are particularly relevant for this thesis.

2.6.1 Data Plane Development Kit

The main goal of DPDK is to provide a framework for fast packet processing in data plane applications [65], by taking complete control of the network driver in user-space. Although the standard interrupt-driven model for packet processing is available in DPDK, one of the main contributions of the framework is that it provides various PMDs that completely eliminate the cost of interrupts, in exchange for higher CPU usage. Hugepages are leveraged to lower the number of TLB misses, and core affinity is set to improve cache locality [12]. Packets are directly copied by the NIC into the memory pool of the DPDK app via DMA. In a virtualised context, it is also important to note that DPDK provides an implementation of the vhost protocol, which allows QEMU to offload packet processing to an external handler when emulating a virtual network device [1]. For this purpose, DPDK also provides a PMD.

2.6.2 eXpress Data Path

There is active debate on whether XDP should be considered a kernel bypassing technique: the original XDP paper [31] does not consider it one, while the eBPF documentation [4] states it could be partial or full kernel bypass depending on the exact setup. From the beginning, XDP was designed to be an alternative to DPDK, one that could both match its throughput and make fewer compromises on ease of integration and security. Instead of moving control of the networking hardware out of the kernel, verified user-space code is attached directly to the network driver, and this code runs before packet processing starts. From within the hook, packets can be redirected to a special socket, the AF_XDP socket, which bypasses the networking stack, and can operate in zero copy mode.

Multiple steps are needed to properly set up the socket. First, the UMEM is created in user-space memory as an array that is divided into frames, each of which holds a single packet [15]. These frames are identified by integer descriptors. For receiving packets, a fill queue and a receive queue are created. When the user-space application places descriptors in the fill queue, it tells the kernel that the corresponding frames are empty, and that new packets can be placed there. The kernel notifies the user-space application of new arrivals by placing the descriptors of the received frames in the receive queue. One socket must be created for each NIC queue, but these sockets can

share one UMEM [48]. We do not use transmit in this work.

One important revision that brought AF_XDP closer to DPDK’s performance was the added option of busy polling [39]. Instead of relying on interrupts and softIRQs, the user-space application can directly drive the NAPI schedule. The application must poll frequently; otherwise, depending on the exact setup, packets could be lost or interrupts could be automatically re-enabled by the kernel.

2.7 Generic Architecture

The generic architecture of a virtualisation-based CPC system that implements forwarding and PRP in software is depicted in Figure 2.5. This is our chosen abstraction, and it is the starting point for the design of the three network stacks we propose.

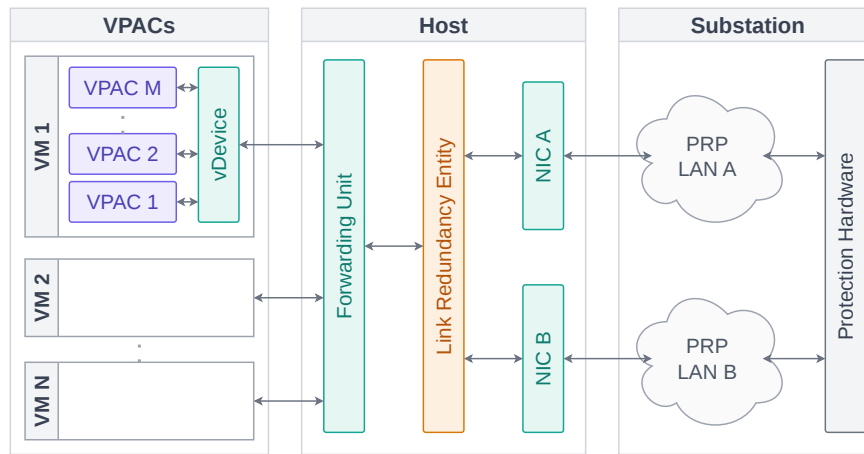


Figure 2.5: The structure of a generic software solution for VPACs.

2.7.1 Substation Grid

This is the logical component that encapsulates all of the grid-specific devices such as sensors and circuit breakers, as well as the onsite networks themselves. The exact topology of the substation networks and the positions of the relevant devices are not taken into consideration, partly because they are not of interest, and partly because the grid management companies mostly keep such details private. However, a few assumptions have to be made regarding QoS, because Operational Technology networks require a controlled, predictable latency and zero packet loss [71]. First, packet loss is assumed to be a rare event, such that at least one of the PRP LANs always delivers

the packet. Second, a latency bound below 1 ms is assumed. Such QoS guarantees can be achieved via TSN or Class C Real-Time Ethernet protocols, or other proprietary methods. The assumptions are consistent with the practices found in IEC 61850 smart grids: Cisco [74] and Siemens [70] offer similar guarantees.

2.7.2 Link Redundancy Entity

This component is responsible for the deduplication, encapsulation and monitoring enforced by PRP. Even though we are restricting ourselves to software solutions, this component can be realised in different parts of the stack. It can be attached to the drivers of the two interfaces, or it can be built on top of them. It can also be built as a kernel module running immediately after the baseline packet processing that executes in the softIRQ context.

Two factors should be taken into account when making the decision: ease of implementation and added cost in terms of CPU time. The low-level solutions are harder to integrate into the network stack, but are more efficient because they drop duplicates early, saving CPU cycles that would otherwise be wasted processing a packet that never reaches the VPACs. The opposite is true of the other solutions. One other option is to cleanly separate the VMs at this level as well, by moving one instance of the Link Redundancy Entity (LRE) into each VM. This option is ruled out by the previous considerations: difficulty of integration is the same as with the host-side counterpart, while the cost would necessarily be higher.

2.7.3 Forwarding Unit and Virtual Devices

The Forwarding Unit delivers the voltage and current values to the VPACs via the virtual devices. Two types of virtual devices are used for this task: the virtio-net device and the Inter-VM Shared Memory (IVSHMEM) device.

VirtIO devices are paravirtualised devices designed to be efficient in virtual environments [78]; the virtio-net device is the virtual network device that abides by the VirtIO specification. IVSHMEM is a specialised implementation of shared memory designed for virtualised environments, which works by emulating a virtual PCI device. It also provides an interrupt mechanism, called a doorbell [42, 37].

2.7.4 VPAC

The Virtualised Protection and Control (VPAC) system is primarily responsible for detecting faults, and secondarily for all other protection and control operations that have low variance in cost and can therefore be placed on the hot path. The standardised

ANSI protection algorithms themselves [5, 19] fall outside the scope of this thesis. When simulation is needed for measurements, they are modelled as fixed-cost compute tasks. When the protection algorithms signal a fault, they transmit the trip message as per IEC 61850.

3 Overview

Before we present the designs and the workflows, the inner workings of the ANSI protection algorithms require clarification. Our analysis of open-source implementations of these algorithms shows that many of them do not directly take current and voltage values from the SV frame as inputs, but rather first “measure” them, that is, transform them into phasors using a running window DFT variant. This suggests two options: one can either deliver the frame as a whole to the VM layer through the virtual network, or first apply the measurement operation in the host, and then save the values in a shared memory region between the host and the VM. Next, we present an overview of the three proposed network stacks and describe the design goals.

3.1 Shared-Nothing

The Shared-Nothing variant has the simplest structure of the three (see the top panel of Figure 3.1). It uses components that are readily available in the Linux upstream kernel, and it preserves receive/transmit symmetry. The PRP kernel module implements in-software redundancy and network monitoring. The virtual network is placed on top of the module to forward accepted packets to the VM. PRP is transparent to the VPACs: they see one virtual network. The entire packet, without the RCT, reaches the VPAC. Therefore, the VPAC is responsible for header parsing and for the measurement operation. This design is used as a baseline for comparisons throughout the thesis, but it is also a viable solution in its own right, because it does not demand any VPAC modification.

3.1.1 Workflow

We present the workflow of this stack variant, based on the top panel of Figure 3.1. We abstract away from the implementation details regarding the kernel networking stack (see Section 2.5).

1. The SV packet arrives at one of the NICs and the system is interrupted to process it. After the kernel allocates the packet-representation data structure, the packet reaches the kernel module.

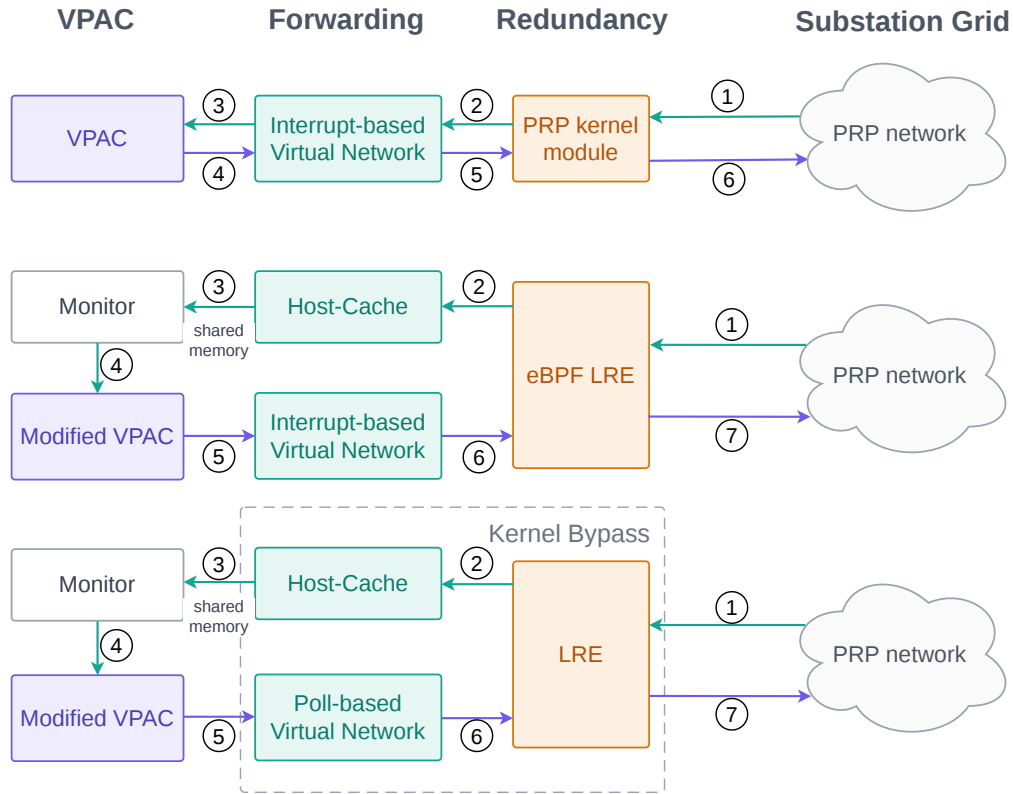


Figure 3.1: Overview of the three stack variants. The green arrows mark the receive path and the purple arrows mark the transmit path. The circled numbers match with the steps of the workflows.

2. The PRP kernel module checks whether the packet is a duplicate, using a sparse bitmap approach. A duplicate is uniquely identifiable through the pair composed of the source MAC address and the sequence number of the RCT. If the packet is a duplicate, it is dropped. If not, it is stripped of the RCT and then passed to the virtual network.
3. The packet is forwarded to the VPAC based on its SV-specific multicast destination address.
4. The VPAC wakes up, applies the measurement and protection operations, and decides whether to trip or not. If so, then a GOOSE packet is transmitted through the virtual network.

5. The packet is forwarded to the PRP kernel module based on its GOOSE-specific multicast address.
6. The kernel module encapsulates the packet with the RCT. It transmits the packet on both physical interfaces.

3.2 AF_XDP-based Host-Cache

This variant strikes a balance between the other two. It makes use of a partial form of kernel bypassing that gives almost full control of packet processing on the receive hot path, while preserving the Linux kernel networking stack for best-effort traffic. The functionality that is provided in the Shared-Nothing variant by the PRP kernel module is moved to eBPF hooks. For this task, we design a new deduplication algorithm that shares similarities with the one in the kernel module. The host-cache is responsible for applying the measurement operation. Modified VPACs access the phasors through shared memory and they are notified of new data by the monitor. In this design, depicted in the middle panel of Figure 3.1, the virtual network only has a function on transmit.

3.2.1 Workflow

The workflow is depicted in the middle panel of Figure 3.1. For the implementation details regarding the eBPF LRE and the AF_XDP setup, see Subsection 5.2.1 and Subsection 5.2.2.

1. A packet arrives at one of the NICs. The kernel-bypassing application drives packet processing and the packet reaches the eBPF program in the driver context.
2. The eBPF program checks whether the packet belongs to critical traffic, and then, whether it is a duplicate. If both checks pass, further processing ensues in the host-cache.
3. The host-cache applies the preconditioning (measurement) operation, and the data is copied to the ring buffer in shared memory that corresponds to the SV stream identifier of the frame.
4. The monitor sees the new data and notifies the VPAC.
5. The VPAC wakes up, applies the protection operation and decides whether to trip or not. If so, then a GOOSE packet is transmitted through the virtual network.

6. The packet is forwarded to the egress eBPF program based on its GOOSE-specific multicast address.
7. The eBPF program encapsulates the packet with the RCT. It transmits the packet on both physical interfaces.

3.3 DPDK-based Host-Cache

This variant leverages full kernel bypass to take control of the physical interfaces in user-space on both transmit and receive. At a structural level, the difference between the two Host-Cache variants is that this kernel-bypassing application also incorporates all forwarding and PRP-related functionality (see bottom panel of Figure 3.1). The interrupt-based virtual network is replaced by the poll-based virtual network through a poll-mode driver that directly fetches packets from the virtual network device.

3.3.1 Workflow

The bottom panel of Figure 3.1 depicts the workflow of this stack variant. We opt for a generic description, emphasising the poll-based model of packet processing. An in-depth discussion of the implementation is the subject of Subsection 5.2.2.

1. A packet arrives at one of the NICs. The kernel-bypassing application drives packet processing through a modified poll-mode driver, and the packet reaches the LRE.
2. The LRE checks whether the packet belongs to critical traffic, and then, whether it is a duplicate. If both checks pass, further processing ensues in the host-cache, and Steps 3–5 proceed exactly as in the AF_XDP variant.
6. The kernel-bypassing application actively checks for new arrivals in the virtual network. It fetches the GOOSE packet and forwards it to the LRE.
7. The LRE encapsulates the packet with the RCT. It transmits the packet on both physical interfaces.

3.4 Design Goals

The design goals are dictated by the needs of substation grids, and by the industry's renewed trend towards scalability. The first two are a direct consequence of the QoS imposed by the IEC 61850 standard, while the last two are preconditions for any

scalable solution. The motivation for proposing three different network stacks instead of one comes from the varying degree to which the goals can be achieved at the same time. For instance, our evaluation reveals that the DPDK-based solution comes closest to deterministic behaviour, while simultaneously scoring lower than the other network stacks in interoperability, because it demands more changes to how VPACs function.

Low End-to-End Latency

After a fault appears somewhere in the substation, the CPC system has to detect it as fast as possible and send a trip message as soon as it is detected. This corresponds directly to the sum of the End-to-End (E2E) latency between the merging unit and the VPAC, the duration it takes for the VPAC to run the fault-detection algorithms and the E2E latency between the VPAC and the circuit breaker. We aim for a total latency bound of 5 ms, but for the purposes of stronger guarantees, we minimise the total latency.

Determinism

Linux, even with the `PREEMPT_RT` patch, is not considered to be 100% hard real-time [68]. Consequently, even if the measured E2E latencies appear to be compliant, certain precautions have to be taken. Jitter, defined as variance in the latencies [81], should be explained with latency decomposition models. If the operations that produce the jitter are understood, and assumptions are made regarding how often they are executed, then the variance-stacking concern can be at least partially dispelled. There are network stack designs that are easier to analyse in such a way, and they are preferable from the standpoint of determinism.

Interoperability

IEC 61850 has been a well-established standard for many years, and as such, there are many hardware devices, protection algorithm implementations or other software services that either adhere to or interact with it. The system is easier to maintain and extend if the network stack is compatible with as many of these pre-existing products as possible.

Low Resource Usage

Because we are examining the viability of core isolation and pinning with respect to VPAC workloads, it cannot be assumed that idle time is filled by other adjacent services. This becomes particularly important when taking scalability into consideration (for

instance, accommodating more feeders [34]). Ways of grouping operations to minimise idle time have to be identified, and VPAC management costs need to be kept low.

4 Design

In this chapter, we continue with the description of the three network stack variants. We also discuss vCPU to pCPU assignment, and briefly compare our solution with hardware alternatives.

4.1 VM Resource Assignment

As stated in the introduction, our goal is to explore the efficacy of CPU isolation and pinning in achieving determinism for the SV-GOOSE critical traffic path. To a certain extent, this simplifies the mapping between vCPUs and pCPUs: each vCPU that hosts VPACs must be pinned to an isolated pCPU with a 1:1 mapping. By isolation we mean that nothing, whether IRQs, user-space applications, or kernel threads, takes CPU time host-side.

However, besides the protection workload, there are other auxiliary services in the software stack, such as logging services. These could be pinned to the same vCPU as the protection workload, to a different vCPU that is itself 1:1 pCPU-backed, or to a vCPU that does not exclusively own a pCPU.

Assume that auxiliary services share a vCPU with the protection workload, and consider the logging service as an example. The logging service receives data from the VPACs, processes it, and writes it to persistent storage through a virtio-blk device. Such an operation triggers VM exits, such as EPT_MISCONFIG, and determining the exact rate at which they are triggered, and at what points in time, is not trivial. More generally, modelling CPU time requires examining every virtual device used by auxiliary services. VM exits are not the only source of uncertainty, because one also has to determine how much time is spent in the kernel for all relevant system calls. Kernel time can be reduced with the PREEMPT_RT patch and appropriate priority assignments, but VM exits cannot.

To avoid VM exits on the protection vCPU, our design separates the two. The VPAC could enqueue the data on a Single-Producer/Single-Consumer (SPSC) queue. Pinned to the other vCPU, a process dequeues and stores the logs in persistent storage, incurring the VM exit without disturbing the VPAC. Multiple auxiliary vCPUs can be pinned to one pCPU host-side, depending on the exact CPU costs.

4.2 Shared-Nothing Variant Components

This section presents the function of the PRP Kernel Module, motivates the choice of using a Linux bridge, and resolves VPAC co-location. The in-depth structure of the solution is depicted in the left panel of Figure 4.1.

4.2.1 PRP Kernel Module

PRP support was first introduced in the kernel in version 5.9, on top of already existing High-availability Seamless Redundancy (HSR) support [49]. HSR is a PRP alternative, also defined in IEC 62439, in which redundancy is achieved via one network with a ring topology, as opposed to the two-network solution. As previously mentioned, the IEC 62439 standard does not enforce a concrete deduplication algorithm: Araujo, Lázaro, Astarloa, et al. [6] list five options and discuss tradeoffs. In version 6.15 (March 2025), the HSR and PRP designs had to be split: because of the switches in the PRP-based network that support L2 prioritisation, frames with larger sequence numbers can overtake frames with lower sequence numbers [41]. This cannot occur under HSR, so the shared design incorrectly discarded both copies of any frame whose sequence number was lower than expected.

The new version used a drop-window approach; Lindner, Shahraki, and Schulz [49] provide a simplified version of the algorithm and describe its problems. The scenario, in IEC 61850 terms, that causes issues is the following: if an MU is connected to the CPC system in LAN A and LAN B under PRP, and LAN A is faster than LAN B by more than 250 μ s, then frame loss on LAN A leads to a discard-both error. This is a serious problem, and it is the reason for which we do not consider a drop-window approach for our design. F. Maurer provided an algorithm that is more robust [54, 55], and it has been merged upstream into the kernel module. The design is based on a sparse bitmap, backed by an xarray. Existing blocks are removed if a new block has to be created and the capacity has already been reached, or if the block is old. Per node, these data structures need about 3 KB of memory. The performance-related cost comes from a per-node spinlock, whose impact is visible in Section 6.3. This is a reliable design of PRP that is maintained upstream and is used as the LRE for this stack variant.

This last algorithm is similar to our design choice for the Host-Cache variants, although ours is simpler, because we can make extended assumptions about the critical traffic metrics.

The kernel module also maintains a node table which can be accessed from user-space, and sends supervision PRP frames as enforced by the standard.

4.2.2 L2 Forwarding for SV

Although many alternatives, such as VALE or Open vSwitch (OVS), have been proposed to replace the Linux bridge in virtualised environments, the bridge performs comparably to OVS, particularly in cases in which forwarding rules are static, simple, and purely L2-based. The foundational OVS paper reports comparable throughput for a baseline configuration with no need for iptables, while OVS consumes more of the CPU [60]. For our SV critical traffic, such simple rules suffice. Each VM has as many entries in the MDB table as there are SV streams it has to service. Multicast snooping should be turned off, and these forwarding rules should be explicitly added to the table to enforce an L2 firewall.

4.2.3 VPAC-to-VM Assignment

A VPAC is a group of processes that work conjointly to service a number of SV streams, and we have already established that the protection workload must be pinned to a vCPU that has complete control of a pCPU. The multiple-vendor assumption is the foundation of virtualised CPC solutions. Considering that, we cannot assume that protection-related computation can be coalesced into one VPAC, so that we could utilise one vCPU at its maximum capacity. One way to attain resource efficiency is to target low-power usage. In that case, the design decision is to give each VPAC its own core, allowing the CPU to enter sleep states if the VPAC is not computing. This approach fails, because one of the consistent findings in the literature is that sleep states and frequency scaling must be disabled in latency-critical networking setups [25, 28, 73]. Therefore, we cannot rely on per-core energy efficiency. The number of physical cores in usage is the important metric, and therefore, attempts must be made to co-locate multiple VPACs in the same VM and on the same vCPU.

The Shared-Nothing stack has a problem in this context: it is not clear how isolation between VPACs can be achieved given that they are reading from the same virtio-net device. One option is eBPF-based socket filtering [16, 51]. With the `S0_ATTACH_FILTER` option, eBPF filter code can be attached to any socket. A monitor creates one socket for each VPAC, attaches filter code to ensure that each VPAC only sees its streams, and then uses the `S0_LOCK_FILTER` option to block later modifications. However, this requires the monitor to spawn the VPACs or pass the file descriptors over UNIX sockets. The Host-Cache variants already require VPAC modification, and they are better suited when modifications are acceptable.

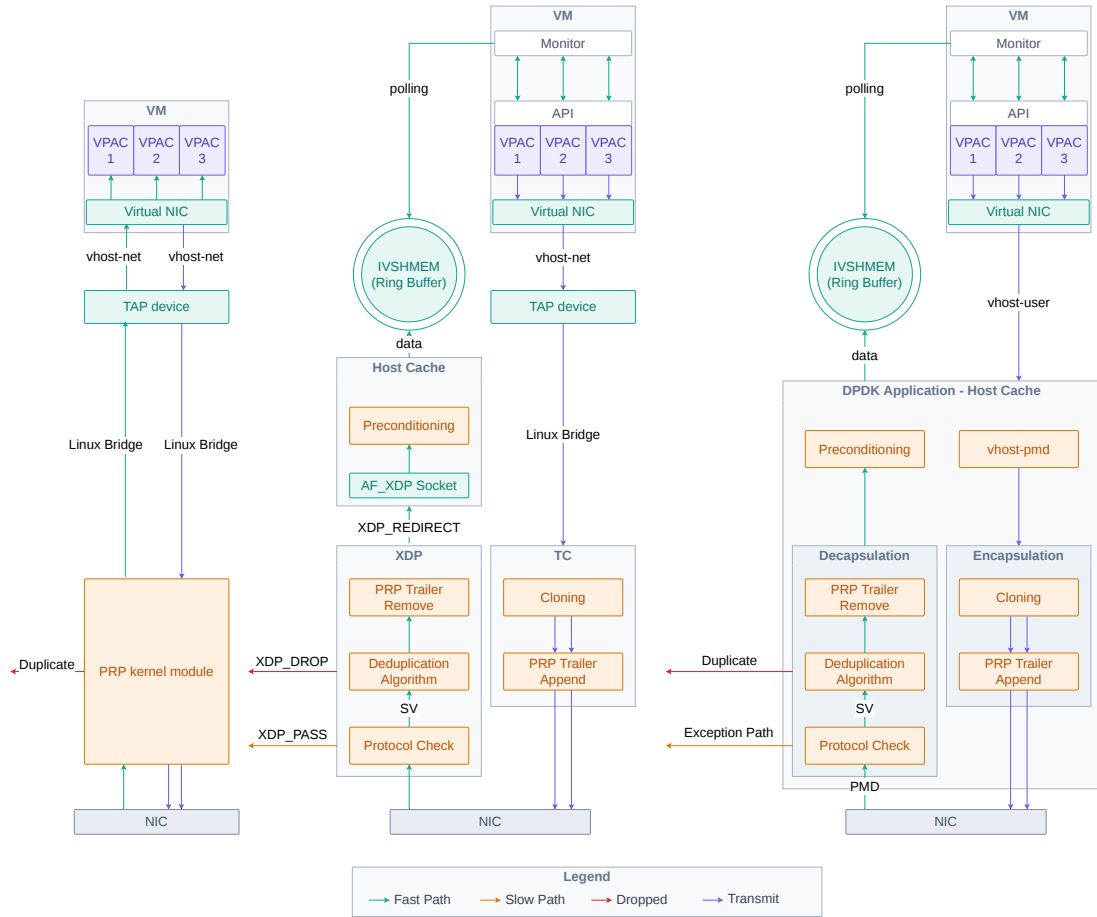


Figure 4.1: In-depth design of the three stack variants: Shared-Nothing (left), AF_XDP-based Host-Cache (middle), DPDK-based Host-Cache (right). The NIC box stands in for both ports.

4.3 Components of the Host-Cache Variants

This section describes the components of the Host-Cache variants: the IPC protocol and its usage by the modified VPAC, the function of the host-cache and our deduplication algorithm. The two solutions are represented in detail in the middle and right-most panels of Figure 4.1.

4.3.1 Modified VPAC

VPACs do not expect to receive SV data over an IVSHMEM region, so they must be modified by their vendors to adapt to a host-cache-based network stack. This reduces interoperability. The transmit API does not change, but VPACs do not receive values through the regular socket API. Instead, we propose a `yield_and_read()` call. When a VPAC finishes processing one stream, it calls `yield_and_read()`, which blocks until the monitor notifies the VPAC about new data. The call returns the identification number of the stream, so that the VPAC directly knows which ring buffer to check. Neither the monitor nor the VPAC has write permissions for the IVSHMEM region. The monitor's role is to keep control of the execution flow in a multiple-VPAC environment. It might seem dangerous to allow the VPAC so much freedom (the freedom of not calling `yield_and_read()` when it should), but the same problem applies to the Shared-Nothing variant. In Section 5.3, we discuss a mechanism to take back control from a VPAC that malfunctions.

Given that phasors are stored in shared memory, we can no longer rely on a virtio-net IRQ to wake a VPAC up if new data has arrived. We identified three possible event notification mechanisms. First, the IVSHMEM device also provides a doorbell; the host-cache writes into the doorbell register when storing new data, triggering an interrupt in the VM. We rule out this option: it reintroduces the interrupts that motivated moving the measurement layer. There is also a second consideration: the IVSHMEM device does not currently possess any interrupt-coalescing capabilities. A proposed rework of IVSHMEM [43] includes the feature, but it has not been merged in QEMU upstream to date. Second, an entire core is reserved for a polling monitor alone, and this monitor sends notifications to the VPACs residing on other cores. The problem is that, according to our measurements, VPACs do not need large amounts of CPU time to complete their tasks. In that case, reserving an entire core for management when the total number of cores needed for protection is small is wasteful. Third, there is one polling monitor placed on each protection CPU, and the polling monitor sends notifications to VPACs that reside on that core. We choose the last solution, and we design an IPC protocol so that the monitor stops polling immediately after sending notifications, and resumes after the corresponding VPACs have finished processing. Abstracting away from the

concrete C implementation, which is the subject of Subsection 5.2.3, we describe the sequence as follows (also see Figure 4.2):

- The polling monitor observes that new values corresponding to an SV stream have been placed in one of the ring buffers. It sends notifications to the VPACs responsible for processing them after setting the value of a shared counter to the number of VPACs that will be woken up. It then waits for a notification from the last VPAC that is going to process the values.
- Each of these VPACs wakes up, completes its processing, and decreases the counter by one. When the counter reaches 0, the VPAC that ran the last decrement notifies the polling monitor.

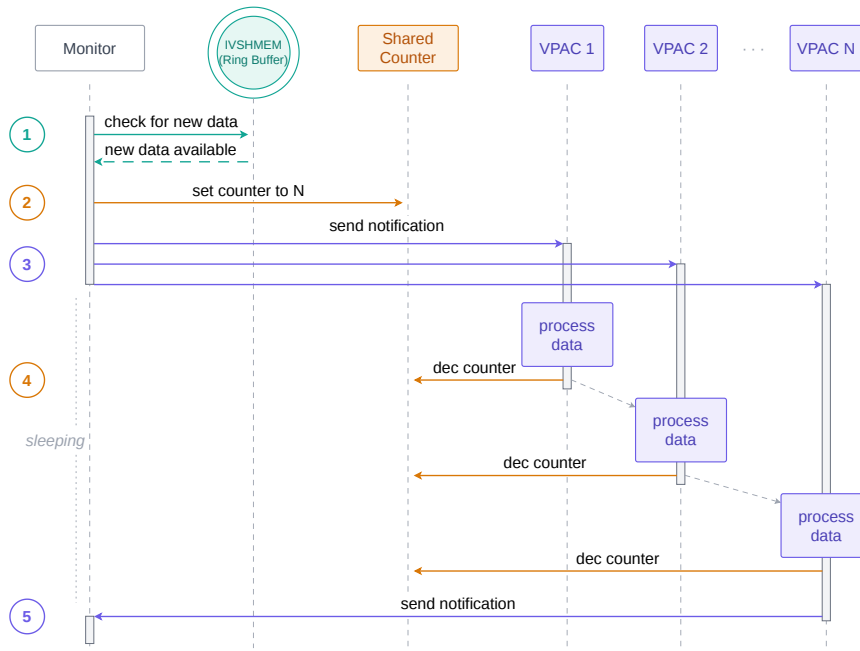


Figure 4.2: IPC protocol diagram.

4.3.2 Host-Cache

The main role of this component, as previously discussed, is to store measured SV data to the per-stream IVSHMEM ring buffers. There are some meaningful differences between the DPDK and the AF_XDP versions. We outline the key design difference,

and Chapter 5 describes the remaining implementation differences. The DPDK host-cache runs the PRP design in user-space. Therefore, it is easy to adapt it or even radically change it. For the AF_XDP variant, we attach the eBPF implementation to the network drivers of the two interfaces via XDP, and are therefore restricted by the limited number of kernel helpers. AF_XDP calls that are concerned with the release of buffers have more overhead than their DPDK counterparts. Therefore, we do not recommend moving PRP processing to user-space.

We describe the structure of the per-stream ring buffer. It contains a write pointer, signalling the first unoccupied entry, and a fixed number of entries containing measured SV data. A typical SPSC design also requires the consumer (in our terms, the monitor) to maintain a pointer that represents the first entry that has not yet been processed, so that a producer stops enqueueing new entries if the consumer is overwhelmed. That behaviour is not appropriate for our use case. If an unexpected event happens in the VM and a monitor and its VPACs are delayed, then the monitor resets to the current write position. Consider that a fault happened at some point in the past during the VM's stall. If the fault is still present, the responsible VPAC detects it by processing current data. If it is no longer present, then there is no reason to trip. This is a consequence of the fact that the true state resides in the host-cache, and VPACs are virtually stateless.

4.3.3 Deduplication Algorithm

We have full control over the PRP deduplication algorithm that runs in the Host-Cache variants. We can restrict ourselves to critical traffic, because PRP management can be cleanly split on a per-device basis. We are particularly interested in SV traffic emitted by MUs, because of the high-throughput nature of the communication. Devices that rarely communicate with the CPC can be handled in less efficient ways, or on the slow path. Out of the algorithm classes discussed in Araujo, Lázaro, Astarloa, et al. [6], drop-window algorithms are not viable, because of the previous considerations. Most of the other alternatives rely on a growing context (stored in a hash table or a FIFO queue), and entries are eliminated based on an aging mechanism. However, on the critical traffic path, we require an algorithm that executes in constant time. Because we know that each MU sends one SV frame approximately every 250 μ s, and under normal conditions the host-cache should receive all of them, implementing data structures that scale up and down depending on load is unnecessary.

We allocate two constant-size arrays per node, both of size 4096, which is one-sixteenth of the sequence space. One of the arrays stores a so-called “epoch”, while the other one determines whether the entry with the same index in the epoch array is valid. Initially, all entries are invalid. The first 4 bits of the sequence number represent

its epoch, while its last 12 bits represent the index in the constant size arrays. When a frame arrives, its source address is used to determine the corresponding pair of arrays, and its epoch and index are computed from the sequence number in the RCT. The frame is accepted if the entry in the epoch array with its index is invalid, or if the entry is valid but the stored epoch is different from its epoch. In the end, the epoch of the new frame is stored, and the entry is set to valid. If no frames are received for 20 ms, then all entries are set back to invalid. A simplified version of the algorithm is depicted in Algorithm 1.

Algorithm 1 Deduplication

Require: packet p , node n , ring size N (we use $N = 4096$)

```

1:  $index \leftarrow p.seq \bmod N$ 
2:  $epoch \leftarrow \lfloor p.seq / N \rfloor$ 
3: if  $\neg n.valid[index]$  then
4:    $n.valid[index] \leftarrow \text{TRUE}$ 
5: else if  $n.epoch[index] = epoch$  then
6:   return DROP
7: end if
8:  $n.epoch[index] \leftarrow epoch$ 
9: return ACCEPT

```

To evaluate the merits of the algorithm, we determine the conditions under which it allows-both or discards-both, and its CPU and memory usage. It allows-both if a duplicate arrives after the epoch entry was changed. That happens when the frame that has the same index but a later epoch arrives before the duplicate, which, at a rate of one frame per 250 μ s, corresponds to a delay of more than one second. The QoS assumptions about the network exclude this scenario. It discards-both if the epoch entry does not change before a full wraparound. The problematic scenario is the following: either of the two frames with a given sequence number arrives, and then all frames with the same index (with the same trailing 12 bits in the sequence number) are lost until a full wraparound. Fifteen frames must be lost in both of the LANs in the span of one wraparound. This scenario is excluded by the guarantees established in Subsection 2.7.1. CPU usage is negligible, but as with the kernel module, the issue is that a per-node spinlock must be acquired.

4.4 Hardware Alternatives

Hardware support for PRP via a RedBox, which is a device that implements redundancy for an end node [14], simplifies the design of all three variants. However, the most

significant difference comes not from a RedBox but from an SR-IOV-capable NIC. The SR-IOV specification defines a type of PCI device assignment that can share a single device among multiple VMs, by configuring a Physical Function (PF) to appear as multiple Virtual Functions (VFs) [79]. Each PF is a standard PCIe function, associated with multiple VFs, while each VF owns performance-critical resources, simultaneously sharing network physical layer processing and packet classification [21].

The hypervisor can expose such a network VF to a VM, mimicking a complete device. Near-native I/O processing is one of the important features, but we are more interested in the data protection and isolation that SR-IOV provides [59]. Neither DPDK nor the Linux kernel stack prevents a crash in host-side networking from rendering all VPACs defenseless: SR-IOV allows for the host-side part of the stack in each variant to be moved into the VM, and contains the fault.

5 Implementation

This chapter contains the description of the low-level implementation of the stack variants, and of the way in which best-effort traffic is separated from critical traffic. We present the network configuration needed to set up the Shared-Nothing variant. Then, moving to the Host-Cache variants, we present the eBPF version of our deduplication algorithm, followed by the libxdp and the DPDK implementations of the host-cache. We also present the eventfd-based implementation of the IPC protocol for the modified VPACs.

5.1 Shared-Nothing Variant Implementation

For this variant, the implementation is a matter of network configuration. We load the kernel module and place it on top of the two physical interfaces as in Listing 5.1. The supervision parameter specifies the last byte of the multicast MAC for supervision frames. `proto 1` corresponds to PRP, while `proto 0` corresponds to HSR. We create a bridge, and we set it to be the master of the PRP interface. For each VM, we instruct QEMU to create a network device abiding by the VirtIO model and to set the master of the corresponding tap device to be the newly created bridge. For isolation purposes, we disable learning and disallow the redirection of multicast frames with addresses that are not in the multicast table. We then add the multicast address corresponding to each serviced SV stream (see Listing 5.2).

```
modprobe hsr
ip link add name prp0 type hsr slave1 eth0 slave2 eth1\
supervision 45 proto 1
```

Listing 5.1: Setting up the PRP kernel module with two physical interfaces, `eth0` and `eth1`.

Host-side, one question remains regarding the number of configured receive queues and the affinity of the corresponding IRQs. Assuming the NIC does not support n-tuple filtering on destination MAC addresses, we recommend one receive queue per physical

```
sudo bridge link set dev $TAP0 mcast_flood off
sudo bridge link set dev $TAP0 learning off
sudo bridge mdb add dev $BRIDGE port $TAP0 grp 01:0c:cd:04:00:00 permanent
```

Listing 5.2: Disabling learning, mcast_flood and adding SV stream 0 to the first VM.

interface, both IRQs pinned to the same CPU. The recommendation is motivated in Section 6.3.

In the VM, we disable transmit interrupts. This implies that completion cleanup happens at transmit or during receive interrupt handling [11]. When the decision was made to enable transmit interrupts by default on the virtio-net device, this was motivated by TCP performance, which is irrelevant for our workload. We refer to Subsection 6.1.5: disabling transmit interrupts allowed us to reduce unpredictable variance at increased loads.

5.2 Implementation of the Host-Cache Variants

In this section, we present details regarding the implementation of the two Host-Cache variants. When needed, we select code snippets that depict our usage of eBPF and of the DPDK framework.

5.2.1 eBPF Implementation of PRP

We previously introduced eBPF as the vehicle that allows us to attach code to the network driver and to the traffic control layer, but there are many other available points in the kernel where eBPF programs can be attached. Like functions, these programs have return values that have different semantics depending on the hook. They are typically written in C and compiled with LLVM; this is the method we used for development [50]. Before the loader attaches the program, the kernel verifies it to ensure that it does not break any critical components. Helper functions are provided by the kernel to safely execute tasks that would otherwise break the sandbox, and eBPF maps provide a way to store and use data between invocations and hooks. Regarding concurrency, the kernel only guarantees that each program invocation has strong CPU affinity [13]. Race conditions remain possible, because multiple invocations can run concurrently on different cores. To avoid race conditions, we use atomic operations when one field has to be updated safely, and spinlocks otherwise.

XDP Deduplication

As input, the XDP program receives the packet data, its length, a pointer to a metadata region, the index of the physical interface on which the packet was received, and the index of the receive queue [62]. We need the packet data and its length for the PRP implementation. The rest of the information is needed to correctly redirect the packet to the AF_XDP application. Three return codes are relevant for us in the XDP hook: XDP_DROP, XDP_PASS and XDP_REDIRECT. We drop duplicates, pass best-effort traffic that we do not handle, and redirect critical traffic to the AF_XDP application. Note that an XDP_PASS return code signals to the kernel to let the packet follow its regular path through the stack.

For our deduplication algorithm, we need two fixed-size arrays per node to store information about the sequence numbers of the previous packets. Optimisations can be made to reduce memory usage. Eight valid entries can be stored in one byte, and, because the epoch is one nibble long, two epochs can be stored in one byte. Therefore, per device, not taking into consideration metadata and padding, we need 2560 bytes. We use this last optimisation in our implementations, so the epoch array is 2048 bytes long, and the valid array 512 bytes long. For this purpose we use two BPF_MAP_TYPE_ARRAYS, each containing a number of entries equal to the number of devices. Each entry of the first map holds one device's epoch array; each entry of the second holds the device's valid array. Additionally, we need one spinlock per device because the critical section contains modifications to both the valid array and the epoch array. The C structs are depicted in Listing 5.3. LIBBPF_PIN_BY_NAME is a flag that ensures persistence across the loader's lifetime.

The algorithm is implemented using bit shifts and masks. If a new arrival is a well-formed PRP frame that belongs to critical traffic and is not a duplicate, then it is redirected to the AF_XDP application.

We attach the program to both of the physical interfaces. It is invoked on receive.

TC Encapsulation

This program is called with a struct `sk_buff` as context [61]. This struct is the main packet representation in the kernel [77]. We use the return codes TC_ACT_SHOT in failure cases (for instance when the helper functions fail), and TC_ACT_OK otherwise. We need to persistently store the sequence number across invocations, and we again use a BPF_MAP_TYPE_ARRAY for that purpose. We use a BPF helper to enlarge the packet in the tail by 6 bytes, because we need the space to store the RCT. We store the size, the LAN, the updated sequence number and the PRP suffix. An atomic addition operation suffices for safe updates of the sequence number. We also need to apply a modulo

```
struct epoch_map_elem {
    unsigned char epoch_ring[2048];
    struct bpf_spin_lock lock;
};
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, NUM_DEVICES);
    __type(key, __u32);
    __type(value, struct epoch_map_elem);
    __uint(pinning, LIBBPF_PIN_BY_NAME);
} epoch_map SEC(".maps");
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, NUM_DEVICES);
    __type(key, __u32);
    __type(value, unsigned char[512]);
    __uint(pinning, LIBBPF_PIN_BY_NAME);
} valid_map SEC(".maps");
```

Listing 5.3: The eBPF maps that were used for deduplication.

operation for the sequence space wraparound, but each instance of the TC hook can apply it locally. Because the sequence number is stored as a 32-bit unsigned integer, there is no risk of undefined behaviour. The sequence number cannot be stored as a 16-bit unsigned integer because the atomic helpers demand a 32-bit unsigned integer.

We attach the program to one of the two physical interfaces (see Figure 5.1). We use a BPF helper to clone the packet and redirect it to the other interface, and afterwards, we change the LAN field and transmit it on the initial one. The physical interface that runs the TC hook is attached to the bridge.

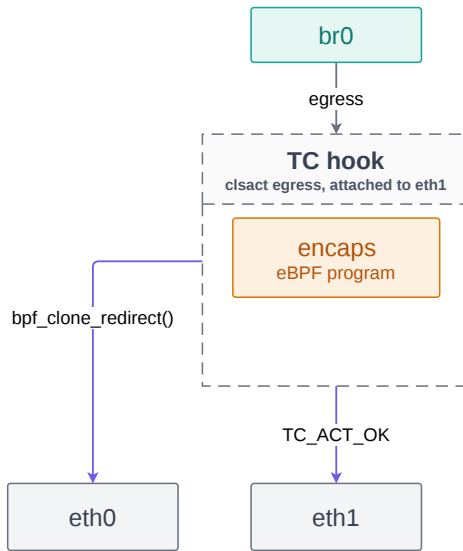


Figure 5.1: The TC program is attached to one of the interfaces, and only that one interface is enslaved to the bridge.

5.2.2 Host-Cache Implementation

The main issue in the design of the Host-Cache variants is CPU usage. A polling-based networking model suits hard real-time better, but requires one core to be reserved for polling host-side. To mitigate the issue, we opt for co-located processing, which means that we do not dispatch the preconditioning operation to another core. We co-locate packet processing and preconditioning on the same physical core, so that, between user-space NAPI invocations (AF_XDP) or PMD calls (DPDK), the host-cache spends its time running SV measurement operations. That time is therefore not wasted; in fact, as the number of streams increases, these host-caches become more efficient at packet

processing compared to the Shared-Nothing variant (see Section 6.3). We now describe the co-located processing implementation for both of the variants.

AF_XDP

We first configure one NIC receive queue per interface, setting the CPU affinity of the IRQs to the core that is running the AF_XDP host-cache. We also need to set the values of two kernel knobs: `napi_defer_hard_irqs` and `gro_flush_timeout`. We previously described the software-interrupt mechanism and the purpose of NAPI. A user-space application can drive the NAPI schedule itself (called “preferred busy polling” mode) by indicating to the kernel that it will poll the socket often enough to process incoming packets [18]. The kernel then deactivates the software-interrupt mechanism. This can be wasteful in periods in which packets are not received, so interrupts can be re-enabled: `napi_defer_hard_irqs` represents the number of empty poll invocations before the kernel unmask interrupts. Our use case is defined by constant-bandwidth and by hard real-time guarantees. We set this knob to the highest possible value, because we never stop polling, and there is no downtime in the protection loop. The other knob is `gro_flush_timeout`. The kernel does not rearm interrupts immediately after the predefined number of empty polls. This knob determines the time that passes before interrupts are re-enabled, so we set it to a low value. We found a value of 5000 ns to yield the best results.

We attach the XDP deduplication program to the two physical interfaces. Without the attachment, the AF_XDP host-cache never receives frames, because they have to be redirected from the XDP hook. We initialise the UMEM as described in Subsection 2.6.2. We share the UMEM between the two sockets we create, one for each physical interface, and we set the preferred-busy-polling flags through `setsockopt`. The rest of main is a poll loop over the two sockets (Listing 5.4).

```
while (running) {  
    for (int q = 0; q < NUM_QUEUES; q++)  
        handle_queue(&queues[q], umem_buffer);  
}
```

Listing 5.4: `handle_queue` is repeatedly called on the two queues. This function drives the NAPI loop by means of a simple `recv` call. The call returns the redirected frames. The SV values are measured, the fill queue is replenished, and then the call returns.

We do not transmit any critical traffic from the AF_XDP host-cache, so we do not

need a completion queue or a transmit queue. This is one of the main differences between this implementation and the DPDK implementation.

DPDK

We start by initialising the Environment Abstraction Layer (EAL), which provides a generic interface to gain access to low-level resources [63], and the mempool, which contains buffers called `mbufs` that are used for packet storage. We map the IVSHMEM region into the process's address space, we configure the two ports, and we set up the connection with the VM for the vhost-user protocol.

We use one physical core, so we loop, processing packets and applying the preconditioning operation immediately afterwards. Three PMD calls are relevant: `rte_vhost_dequeue_burst` to process frames transmitted from the VMs, `rte_eth_rx_burst` to process frames that arrive on the physical ports, and `rte_eth_tx_burst` to transmit frames on the physical ports [69]. In each iteration, we first check for new arrivals from the VM. If we find packets, we encapsulate them according to PRP and we transmit them through the two physical ports. Then, we check for new arrivals on the two physical ports. If we find packets, we drop the duplicates, and we run the preconditioning operation on the remaining packets. We choose the physical port receive pipeline (Listing 5.5) to exemplify usage of DPDK calls. As arguments, `rte_eth_rx_burst` requires the port and the queue identification number, an array of `mbuf` pointers that is used to store the location of the packets in memory, and the maximum number of packets that should be received at once. Note that some drivers require this number to be a multiple of 4, and some require it to be a multiple of 8. After processing the packet, we free the space it occupies with `rte_pktmbuf_free`. This is unnecessary when transmitting the entire packet, because DPDK handles the transmit completion routine for the programmer. The deduplication algorithm is again implemented using bit shifts and masks.

A limitation is given by the one-core restriction, which might not suffice for a large substation. The problem is not the packet rate itself, since Høiland-Jørgensen, Brouer, Borkmann, et al. [31] report that DPDK is capable of sustaining a throughput of more than 20 Mpps on one core, and our deduplication algorithm is not an expensive compute operation. Legacy substation grids use 100 Mbit/s Ethernet links, and the current standard is 1 Gbit/s. Assuming 1 Gbit/s, and considering that SV frames usually contain 1 ASDU, the link is saturated on the order of 1 Mpps, which is well below DPDK's capacity. However, the preconditioning cost might be an issue: 1 Mpps corresponds to about 128 SV streams assuming PRP, and if the per-frame processing cost, adding in other factors such as cache pressure, is in the single-digit microseconds per stream, then processing might not fit on one core. NIC capabilities and DPDK's

`rte_flow` [66] combined offer one solution. The work is split based on the destination MAC addresses of the packets, which correspond directly to streams. Multiple DPDK host-caches are deployed, each handling a statically assigned pool of SV streams. Because data delivery is completely split on a per-stream basis, there is no concurrency, and our one-core solution generalises to this configuration.

```
uint16_t n;
if ((n = rte_eth_rx_burst(port_phy1, 0, bufs, BURST_SIZE)) > 0) {
    for (size_t i = 0; i < n; i++) {
        if (dedupe(bufs[i], prp_state)) {
            precondition(bufs[i]);
        }
        rte_pktmbuf_free(bufs[i]);
    }
}
```

Listing 5.5: The DPDK receive pipeline.

5.2.3 Eventfd IPC Protocol

We need a lightweight notification mechanism for the monitor to wake VPACs up when it detects new data, and for the VPACs to wake the monitor up when they are finished processing. We use the eventfd synchronisation primitive for both tasks. The monitor creates one eventfd per VPAC, which is used for new-data notifications, and one eventfd for synchronisation. It shares two eventfds with each VPAC: the VPAC's own data eventfd and the global synchronisation one. The eventfds can be shared with the VPACs by forking, or by communication over UNIX domain sockets. For our proof of concept, we used the former mechanism. When noticing new data for a stream, which happens when there is a mismatch between the monitor's internal pointer and the pointer stored in the buffer, the monitor writes to the eventfds of the VPACs that service the stream. They know which data to fetch, and they can start processing the new data. When a VPAC has finished processing, it calls `yield_and_read` (Listing 5.6), in which the VPAC writes to the synchronisation eventfd if it was the last one to process. That in turn wakes the monitor up. The VPAC then goes back to sleep, waiting for new data again. This repeats in a loop.

We previously mentioned that protection costs per stream are low. The eventfd notification mechanism itself is lightweight, but context switches are not. In Chapter 6 we estimate the per-stream cost of the IPC protocol to be around 3.26 μ s. That means

```
uint64_t yield_and_read(Control *ctrl, int vpac_efd, int sync_efd) {
    uint64_t u = 1;
    volatile atomic_int *counter = &(ctrl->counter);
    int old_counter = atomic_fetch_sub(counter, 1);
    if (old_counter == 1) {
        write(sync_efd, &u, sizeof(uint64_t));
    }
    efd_read(vpac_efd, &u);
    uint64_t stream_id = u - 1;
    return stream_id;
}
```

Listing 5.6: The `yield_and_read` call in C excluding error checks. The monitor writes `stream_id + 1` to the eventfd to avoid the case in which the `stream_id` is 0, because writing 0 does not trigger a notification. That is why the VPAC must subtract 1 from the value it reads. Control is not the IVSHMEM region, but another VM-local shared memory region for VPAC-Monitor communication. `efd_read` is just a wrapper around the read system call.

that the management cost is higher than the protection cost. This is unavoidable in the worst-case scenario, and the Shared-Nothing stack itself suffers from the same problem in the multiple-VPAC setup, because for each received packet a context switch to another VPAC is needed given the worst-case packet arrival pattern.

However, the NAPI mechanism itself suggests an optimisation that can reduce the number of context switches. If a VPAC handles protection for many streams, and it is woken up by the monitor because new data has arrived from one of them, the VPAC can also check all of its other streams for new data before yielding. If it finds new data from any of the other streams, it processes it, and it notifies the monitor so that it is not woken up for data it has already processed. It does so by sharing its internal ring buffer pointers with the monitor over a VM-local shared memory region. This optimisation can have a considerable impact: each VPAC typically processes more than one stream. The MUs have synchronised clocks, and they package and send the values at approximately the same time. Therefore, under normal conditions, the frames reach the host-cache simultaneously. Because the other host-cache costs are negligible, the inter-arrival gaps as the VM sees them are determined by the measurement operation, whose cost is small enough for this optimisation to be meaningful. The optimisation reduces the average management cost, but we do not recommend placing heavier protection workloads in each VM assuming that it does so in the worst case as well. It

is difficult to guarantee inter-arrival gaps on the order of low microseconds even in a TSN-grade network.

5.3 Watchdog

Regardless of the variant, a mechanism has to be implemented so that VPACs that do not behave correctly are identified if several are co-located on one core. For the Host-Cache variants, we attempted to solve the problem by wrapping the eventfd read of the monitor with an `epoll_pwait2` call. In that case, the read blocks for a predefined amount of time. If control is returned to the monitor before the timer runs out (because a VPAC wrote to the eventfd), then regular execution ensues. If the timer does run out, then the monitor can identify which VPAC is causing the stall. We observed higher latencies when modifying the code in this way, because of the high-frequency arming operation. An alternative arms the timer at a lower frequency, for instance once every 10 ms. Note that this type of mechanism can be implemented because there is a per-VM control shared memory region between the VPACs and the monitor (or the watchdog). Before a VPAC reads values, the API calls can also enforce a write to this region, so that stalls are identifiable.

It is not clear how a lightweight detection mechanism could be implemented given the Shared-Nothing variant without modifying the VPACs. This is another point against having multiple VPACs in one VM in the Shared-Nothing variant, added to the previous considerations of Subsection 4.2.3.

5.4 Handling Best-Effort Traffic

As discussed in Section 4.4, complete isolation of a VM from the rest of the system with respect to networking can only be achieved with an SR-IOV-capable NIC. However, from a performance standpoint alone, ethernet-based n-tuple filtering can separate critical traffic receive queues from best-effort traffic receive queues if the NIC supports the feature. Not many NICs do, so we assume that we have to filter out best-effort traffic in software. We also assume that the rate of best-effort traffic is lower than that of critical traffic, at most reaching the combined rate of a few SV streams.

There is an inherent tradeoff between ease of integration and determinism. Given the assumptions made above, if the kernel networking stack is kept, handling best-effort traffic does not require any extra effort. These packets take the same path, through the PRP kernel module and through the bridge, the only difference being that they are forwarded to a different destination. As a precaution, VMs that host VPACs receive virtual network devices dedicated to best-effort traffic. As such, the IRQ corresponding

to the best-effort traffic virtual network device ought to be given a lower priority compared to the IRQ corresponding to the critical traffic virtual network device.

Best-effort traffic does not pose a problem for the AF_XDP-based stack. XDP was designed to provide a DPDK alternative that is easy to integrate, and that could approach DPDK's throughput capacity [31]. In a "run-to-completion" [40] AF_XDP architecture, where interrupts are enabled, all considerations from the previous paragraph remain valid. Best-effort traffic is passed up the stack with the appropriate XDP return code, taking the standard path. However, as documented before, our choice was the busy polling AF_XDP architecture. Here, best-effort traffic is processed when the AF_XDP application is actively polling the socket. Consequently, the application must poll at the required frequency to not indefinitely delay best-effort traffic. This is not a problem: the best-effort traffic shares the receive queue with the critical traffic, and we poll often enough for the critical traffic. Therefore, no issues ensue as long as the rate of best-effort traffic is as assumed.

DPDK is difficult to integrate into existing software stacks: it makes traditional network monitoring and debugging tools unusable [9]. In addition, MMS runs on TCP/IP [52], and DPDK does not have a built-in implementation of TCP/IP. DPDK already provides a solution via the so-called exception path [32]. Packets are injected into the Linux network stack through a tap device using the vhost protocol, and regular processing resumes on top of it. Consequently, the DPDK application can use the exception path to filter out packets that do not have the ethertype of SV.

6 Evaluation

This chapter comprises the evaluation framework and the concrete results of the measurements. We begin with two preliminary considerations.

First, there are sections in this chapter where our interest is concentrated on the host-side stack. To better isolate the host-cache, we use a variant of the Host-Cache VPAC that induces less overhead. In a setup with one VPAC, the role of the monitor can be taken up by the VPAC. It can scan the ring buffers on its own, without a handoff mechanism. When using this VPAC type, we explicitly refer to it as the no-monitor setup.

Second, when increasing the number of VPACs, we observed insignificant effects up to three VPACs, and small but noticeable effects at larger numbers. Because of these findings, except for Subsection 6.5.1 where we discuss multiple VPAC measurements, we restricted ourselves to one. In a real deployment, the indirect cost of a context switch might rise with increased VPAC numbers, but this is a matter we acknowledge as an unexplored direction (see Subsection 6.1.5).

In the following sections, we model the capacity of the protection vCPUs and we present a latency decomposition model for the DPDK variant. We measure the round-trip time (RTT) of the three network stacks. To better compare them, we measure performance on a per-layer basis. For the purpose of generalisation, we focus on the management costs induced directly by the networking stack, rather than the protection costs that are variable. At the end of the chapter, we focus on validating the models using measurement data.

6.1 Models

To begin, we provide an overview of the models, motivating their importance with reference to two questions that are hard to solve by means of measurements alone. After listing our assumptions and the notation we use, we describe the VM capacity models and the worst-case burst model for the DPDK variant.

6.1.1 Overview and Motivation

There are two questions that warranted going beyond the measurement of E2E latencies.

First, one of the negative aspects of our one-to-one vCPU to pCPU approach is that it is prone to wasting CPU time. We previously argued that the cost of allocating more CPUs cannot be coupled with power saving to reduce resource waste in general, because latencies have high fluctuations if frequency scaling and sleep states are not disabled. Therefore, we sought methods to determine the capacity of each protection vCPU, because this allows engineers to increase the per-vCPU protection load without breaking QoS. By capacity we mean the number of SV streams that can be serviced by one protection vCPU, given some average protection cost per stream, and assuming no data loss in other layers.

Second, even a substation network with TSN-grade QoS can have many concrete realisations. PRP alone already allows for the LANs to have different switch topologies. We, however, are limited by our test bench. Consequently, we cannot test all configurations, although we need to determine the worst-case latencies.

To complement the measurements and better answer these questions, we develop models for vCPU capacity and the worst-case latencies under substation network stalls. We base our models on an abstraction of the system with CPUs as execution units with multiple task queues. The DPDK-based variant yields the clearest example: the CPU of the host-cache and the vCPUs tasked with protection are such execution units. Each protection vCPU has one queue with the data within each SV frame defining a task. The host-cache has one receive queue, with two task types, processing frames that are accepted and processing frames that are dropped. It also has a transmit queue, whose task involves dequeuing a frame through vhost-user, encapsulating it according to PRP, and sending it on both physical interfaces.

We already defined capacity for protection-related execution units. Similarly, the host-cache execution unit's capacity is defined in terms of the number of SV streams it can handle on average, assuming no losses in the other layers. Within our modelling framework, this means that the queues associated with the execution units cannot grow indefinitely. We thus obtain one system-wide capacity condition by combining the two.

However, that is not sufficient to guarantee that the latency bounds hold. Under normal conditions, the substation network delivers the synchronised SV frames almost instantly, but this is not the worst case. Let $T = 250 \mu\text{s}$ denote the SV sampling interval, and suppose the network guarantees a bounded on-the-wire latency of $D = K \cdot T$ between source and destination. Then the following sequence becomes possible. A stall in the network delays a window of SV traffic sent at time t by $K \cdot T$. Consequently, because all SV traffic shares the same priority and is therefore not reordered, the SV frames sent at time $t + T$ are delayed by $(K - 1) \cdot T$. In general, the SV frames sent at time $t + i \cdot T$ are delayed by

$$d_i = (K - i) \cdot T, \quad 0 \leq i \leq K. \quad (6.1)$$

The conclusion is that the system has to be capable of handling $(K + 1)$ windows of SV frames under the worst-case substation network stall. This is the second condition that must be met to guarantee that the trip-related bounds hold. In the model, this corresponds to a makespan derivation given the load described above, which is the subject of Subsection 6.1.4.

Table 6.1 summarises the symbols used throughout this section. They denote averages in the capacity-related subsection, and worst cases in the worst-case burst subsection.

6.1.2 Assumptions

SV is a constant-bandwidth protocol, but the number of SV streams in a substation grid, as discussed in the introduction, is not fixed. However, substation grids are not prone to the type of dynamic reconfiguration that defines cloud environments. Connecting and disconnecting feeders are not automatic operations, because they involve interaction with the hardware. We can therefore assume that the number of SV streams stays fixed during execution. Given that each MU sends at a rate of one frame per $250\ \mu\text{s}$, we can now state the capacity conditions. On average, per execution unit, the processing of all SV streams must not take more than $250\ \mu\text{s}$. For the protection-related execution units (the vCPUs), this includes the GOOSE trip. Because GOOSE is used for emergency communication, we generally assume at most one trip per $250\ \mu\text{s}$. For the host-cache execution unit, it includes the forwarding of GOOSE trips and discarding duplicates, besides the per-accepted SV frame measurement operation.

The substation network is assumed to guarantee near-zero packet loss and bounded latencies of 1 ms. All models also assume that each stream is processed by one VPAC, primarily because it simplifies the notation. The models are easily extendable to fit the more general case.

We do not consider the optimisation that allows modified VPACs to process multiple streams before yielding. It performs at least as well as the unoptimised version in all cases, but predicting its CPU usage is not possible without further assumptions that are restrictive. It also further simplifies the parameters we have to consider, because the number of VPACs ceases to be important. A discussion of the constant costs and the measurement methodologies is the subject of Subsection 6.1.5.

6.1.3 Capacity Models

The unmodified VPAC receives frames over the virtio-net device, it runs the protection algorithms, and it transmits at most one GOOSE trip for every window of SV traffic. The receive network IRQ is pinned to the protection core, so all packet processing costs enter the capacity model. This is the capacity condition, which is directly related to

Symbol	Unit	Description
M	—	Number of SV streams
T	μs	Time window defined by the distance between SV frames; generally equal to $250\mu\text{s}$
D	μs	Bounded on-the-wire latency
K	—	Number of windows that fit in the delay D
t_{IPC}	μs	The cost of the two user-space-to-user-space context switches, two eventfd reads and two eventfd writes
t_{scan}	μs	This is the cost of the monitor scan and of the VPAC scan
t_{handoff}	μs	$t_{\text{scan}} + t_{\text{IPC}}$
$t_{\text{protection}}$	μs	Protection cost per stream; in practice, differs from stream to stream, with small variations
$t_{\text{precondition}}$	μs	Preconditioning cost per stream; in practice, it differs from stream to stream, with small variations
$t_{\text{process-packet}}$	μs	Complete cost of processing one virtio-net receive inside the VM: interrupt handling, softIRQ processing, <code>sk_buff</code> allocation, and delivery up to the user-space socket
t_{send}	μs	Cost of a packet send system call; self-contained because of our choice to disable transmit completion interrupts in the VM
$t_{\text{SV-accept}}$	μs	Cost of receive, deduplication and preconditioning in the host-cache
$t_{\text{SV-drop}}$	μs	Cost of receive and deduplication in the host-cache
$t_{\text{GOOSE-PRP}}$	μs	Cost of a vhost-user receive, encapsulation, and send on both physical interfaces in the host-cache

Table 6.1: Notation used in the capacity and latency models. Some other notation is introduced when needed in the relevant section.

CPU usage

$$M \cdot (t_{\text{process-packet}} + t_{\text{protection}} + t_{\text{precondition}}) + t_{\text{send}} < T \quad (6.2)$$

In the modified VPAC setup, there is no receive-related packet processing, but there is an eventfd-triggered context switch from the monitor to the VPAC, and then another one back to the monitor. This is the corresponding capacity condition

$$M \cdot (t_{\text{handoff}} + t_{\text{protection}}) + t_{\text{send}} < T \quad (6.3)$$

with $t_{\text{handoff}} = t_{\text{scan}} + t_{\text{IPC}}$. Note that we wrote t_{send} to avoid complicating the condition, but as Subsection 6.1.5 makes clear, the cost of a vhost-net send is different from the cost of a DPDK vhost-user send.

The DPDK host-cache execution unit allows for a similar condition. Per window, it accepts one frame per stream, it discards one frame per stream, and it forwards at most one GOOSE trip

$$M \cdot (t_{\text{SV-accept}} + t_{\text{SV-drop}}) + t_{\text{GOOSE-PRP}} < T \quad (6.4)$$

An important aspect is that delays in other layers do not affect the capacity of protection-related execution units. Losses, however, can occur if the packet-storing buffers across the stack are not sized correctly.

The DPDK Host-Cache system is stable when Equation 6.4 and Equation 6.3 hold simultaneously. We call L_V and L_H the per-window loads of a protection vCPU and of the host-cache, respectively.

Subtracting the two loads gives

$$L_V - L_H = M (t_{\text{handoff}} + t_{\text{protection}} - t_{\text{SV-accept}} - t_{\text{SV-drop}}) + (t_{\text{send}} - t_{\text{GOOSE-PRP}}), \quad (6.5)$$

so the protection vCPU is the bottleneck regardless of M if $t_{\text{handoff}} > t_{\text{SV-accept}} + t_{\text{SV-drop}}$ and $t_{\text{send}} > t_{\text{GOOSE-PRP}}$. Both inequalities hold: t_{handoff} is dominated by the context switches and is estimated at roughly $3.26 \mu\text{s} + t_{\text{scan}}$ (Subsection 6.1.5) and t_{send} includes at least one descriptor cleanup, which raises it slightly above $t_{\text{GOOSE-PRP}}$. We take $L_V > L_H$ as given in Subsection 6.1.4 but a similar analysis can be developed for $L_V < L_H$.

Excluding intermediary results that are used in Subsection 6.1.4, the formula of interest is Equation 6.3. Because t_{handoff} and t_{scan} can be measured, it helps with determining the number of streams that can be serviced by one VM given fixed protection costs. Such considerations are the subject of Subsection 6.5.1.

6.1.4 DPDK Variant Worst-Case Burst Model

For now we assume one protection vCPU. We generalise to multiple protection vCPUs at the end of the subsection.

If $D = K \cdot T$, then the worst-case burst presents the host-cache with $K + 1$ instances each of M accept tasks, M drop tasks, and one GOOSE-PRP task, and the protection vCPU with $K + 1$ instances of M processing tasks and one send task. Writing A_{ij} for the accept task of stream i in window j , and similarly for the others, the precedence constraints are

$$A_{ij} \rightarrow P_{ij}, \quad P_{ij} \rightarrow S_j, \quad S_j \rightarrow G_j,$$

for all i and j : a sample cannot be processed before it has been accepted and written to the ring buffer, a trip cannot be issued before all of the window's samples have been examined, and the host cannot encapsulate a trip it has not yet received. The structure is a two-machine flow shop with reentrance, since G_j returns to the host-cache after S_j completes in the VM.

Since $L_V > L_H$, the protection vCPU is idle only until the first sample of the burst has been accepted. By (6.5) and the considerations after it, $A_{i+1,j}$ completes before P_{ij} does, so no further idle period can open, and the vCPU is busy for $(K + 1) \cdot L_V$ thereafter. As such, we obtain

$$\text{Makespan} = t_{\text{SV-accept}} + (K + 1) \cdot L_V + t_{\text{GOOSE-PRP}}. \quad (6.6)$$

To obtain the maximum possible latency, we also add the maximum delay on receive and the maximum delay on transmit. Note that the very last context switch that returns control to the monitor comes after the last send, so it is part of the makespan but not part of the latency. Therefore we obtain

$$\text{Latency} = t_{\text{SV-accept}} + (K + 1) \cdot L_V - t_{\text{context-switch}} + t_{\text{GOOSE-PRP}} + 2 \cdot D. \quad (6.7)$$

If we set $T = 250 \mu\text{s}$, and $D = 4 \cdot T = 1 \text{ ms}$, we therefore obtain

$$\text{MaxLatency} = t_{\text{SV-accept}} + 5 \cdot L_V - t_{\text{context-switch}} + t_{\text{GOOSE-PRP}} + 2 \text{ ms}. \quad (6.8)$$

If there are multiple protection vCPUs, the worst-case latency is obtained by assuming that the host-cache tasks on which a protection vCPU's tasks depend are the very last in the receive queue of the host-cache. Therefore, we obtain

$$\text{MaxLatency} \leq 5 \cdot L_H + 5 \cdot L_V - t_{\text{context-switch}} + 2 \text{ ms} \quad (6.9)$$

For $K = 0$, we obtain the latency decomposition model.

6.1.5 Estimation of Constant Costs

Context Switches

In our context switch estimation methodology, we follow C. Li, Ding, and Shen [47]. The authors identify two types of context-switch-related costs. The direct cost comes from dispatching, scheduling, TLB reloading and CPU pipeline flushing. Indirect costs are generated by the cache contention between processes. We are more interested in the former, because it represents the highest cost in our host-cache VM capacity model. The authors further propose a method for the estimation of the direct cost. Two processes repeatedly send each other 1-byte messages via pipes in a round-trip manner, generating two context switches, two reads and two writes per round of communication. This is measured for a fixed number of iterations. Further, the cost of the pipe operations is measured independently, so that they could be subtracted from the elapsed time from the previous measurement to isolate the direct context switch cost.

Our methodology is shorter. We replace pipes by eventfds, and then we keep the elapsed time of the two process round-trip measurement unchanged. It contains two context switches, two eventfd reads and two eventfd writes, which is precisely the cost of our IPC protocol. We are most interested in the average, which is approximately 3.26 μ s. This is obtained by running the experiment ten times with 100 000 samples, by timing before and after the entirety of the run. We also need the entire distribution, so, in a separate run, we time before and after each instance of the IPC protocol. This introduces more overhead because `clock_gettime` is called twice for each IPC protocol instance, which can be observed in the 3.33 μ s mean (see right-most panel of Figure 6.1). When we need the average, we use 3.26 μ s. When using the entirety of the distribution, as in Subsection 6.5.2, we shift the distribution such that its average is approximately equal to 3.26 μ s, removing the effect of the `clock_gettime` calls.

Virtio-net Send

One of the advantages of disabling completion interrupts with `virtio_net.napi_tx=0` is that the cost of the send system call becomes self-contained. During a measurement run, we also time the system call to obtain the cost. The cost of the vhost-net send is different from the cost of the DPDK vhost-user send. A kick is needed to wake the vhost-net thread up when new packets are transmitted from the VM. This wake-up is administered during a VM exit. There is no need for such a mechanism when DPDK implements the vhost protocol. The results of the measurement are in the first and second panels of Figure 6.1.

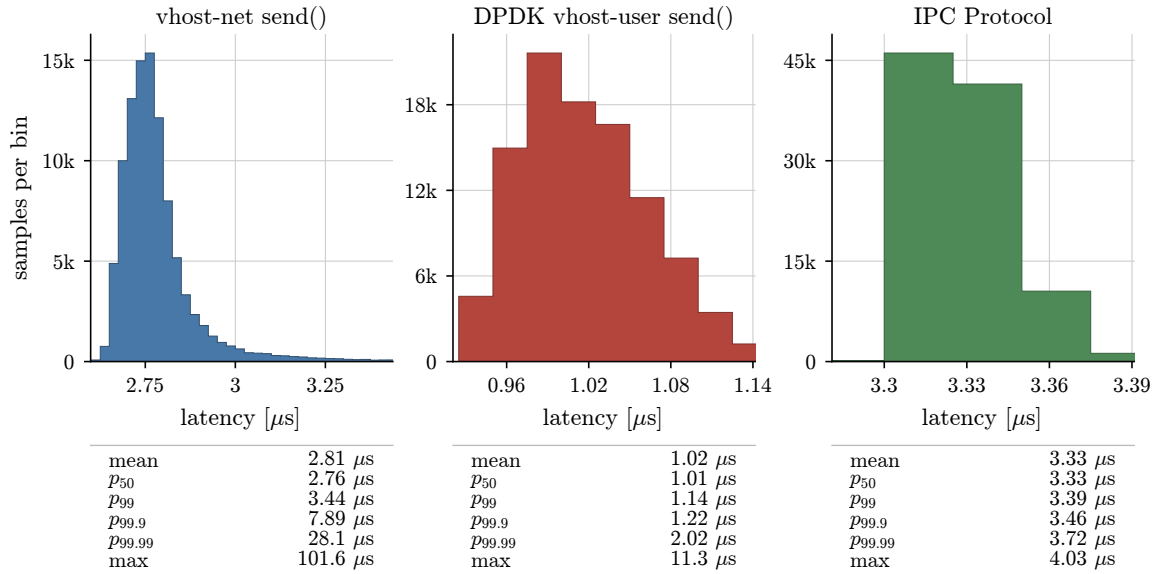


Figure 6.1: Histograms of the costs up to the 99th percentile, with relevant statistics including the deep tail below.

Recommendation #1: Completion interrupts

Disable completion interrupts on the virtual network device by adding `virtio_net.napi_tx=0` to the VM kernel parameters.

As with the context switches, the average is the relevant metric for the capacity models. However, in Subsection 6.5.2 we slightly modify our worst-case burst model to enable validation. There, we need the entire distribution of the cost of the IPC protocol, and of the send system call.

6.2 Measurement Setup

This section describes our load generation method for SV streams, the concrete test bench including hardware devices, and the configuration of the different software and hardware components. The subsection on configuration is particularly important, because it contains some concrete steps that lowered latencies.

6.2.1 Load Generator

For load generation, we used a custom DPDK application. It took direct ownership of two ports: one representing PRP LAN A, the other PRP LAN B. For all measurements but the worst-case burst model validation of Subsection 6.5.2, we simulated SV by sending frames from one or multiple streams every 250 μ s. We did not use hardware timestamping, so, to lower the measurement overhead, we reserved 1 pCPU to generate the load and 1 pCPU to receive the simulated trip. The first received frame was considered for the measurement, and the PRP duplicate was discarded. Because we needed large sample counts, and because our worst-case assumption is one GOOSE trip per 250 μ s, we coordinated the VPACs during the measurement to only trip once every window, after all SV streams had been processed.

6.2.2 Test Bench

Each of the machines (the load generating machine and the device under test) has an Intel Xeon Gold 5317 CPU (12 physical cores, processor base frequency 3.00 GHz) and a dual-port Intel Ethernet Controller X550 (10 GbE data rate per port). The machines are connected over both ports via RJ45 connectors, providing the base for a simplified PRP setup. There are no switches between the machines, and no administration network noise affects the measurements. The test bench is illustrated in Figure 6.2.

The host OS was NixOS with Linux 7.1.4 on the device under test and 6.18.39 on the load generator. The kernel version of the device under test is particularly important, because the kernel module update is a relatively new addition, as discussed in Subsection 4.2.1. The guest OS was Ubuntu 24.04.4 LTS.

6.2.3 Configuration

There are some configuration details that had an impact on our measurement results. Disabling frequency scaling and sleep states is important for reproducibility. Both the E2E latencies and the CPU usage microbenchmarks that we introduced in Subsection 6.1.5 degraded without these settings. We also disabled hyperthreading. We enforced CPU isolation through kernel command line arguments, and we used `systemd` to ensure that no user-space or kernel processes (except for the ones with fixed CPU affinity, such as migration) consume CPU time. We pinned all irrelevant IRQs away, onto the housekeeping cores, both in the host and in the VM.

We always pinned the QEMU emulator to a separate, isolated core. For both the Shared-Nothing and the AF_XDP variants, the vhost-net thread's wake-up penalty is important. Pinning the emulator did not have a visible impact on the DPDK variant, but we kept the pin nonetheless, for symmetry. That is consistent with the information

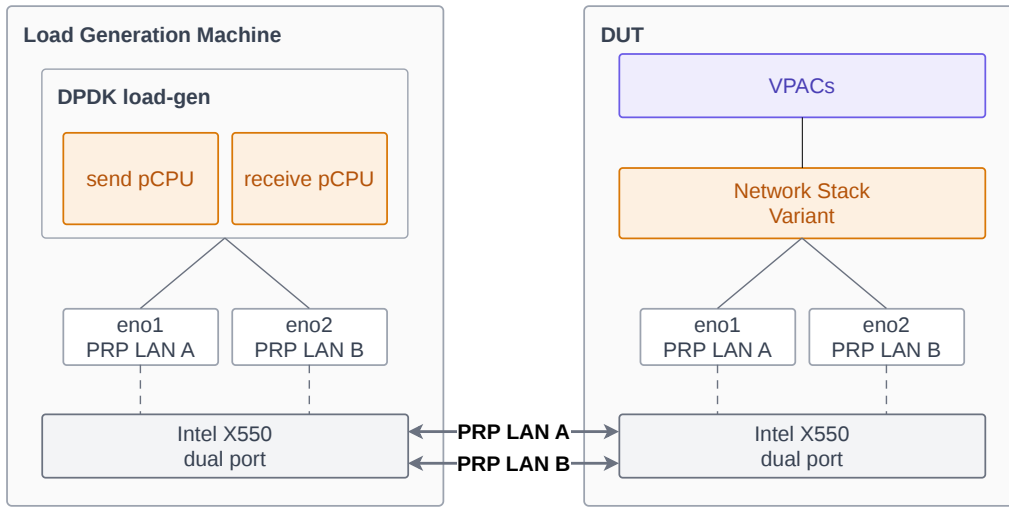


Figure 6.2: A diagram of the test bench, depicting the load generation machine, the device under test, and the PRP setup.

we already have about the stacks: IVSHMEM data sharing completely bypasses the vhost protocol.

We used 1 GB hugepages to back the DPDK applications. 2 MB hugepages were not evaluated. Also, in the DPDK host-cache setup, the VM’s memory must be backed by 1 GB hugepages for performance reasons [67]. Vhost-user requires the VM’s memory to be shared. The fallback mechanism for shared memory if the user does not explicitly specify that hugepages are to be used is `memfd` [56], which added hundreds of microseconds of latency in the worst-case scenario. If vhost-user is not enabled, shared memory is not a requirement for the VM. We have attempted to reduce latencies by backing the VM’s memory with hugepages for the other variants, but we did not observe significant differences.

Recommendation #2: VM memory backing

Back the VM’s memory using 1 GB hugepages in the DPDK variant.

We disabled interrupt throttling on both NICs. In general, this is recommended for latency-sensitive use cases, but it is also required for the measurement in Section 6.3: the methodology is correct only if interrupt throttling is disabled.

6.3 Host-side CPU Usage

Research Question

RQ 1 *Can the host-side components of each network stack process the packets of 128 SV streams within the 250 μ s window, and how much CPU time do they leave for other work?*

There are two aspects that make this measurement important. First, we want to know whether or not all proposed stacks can process up to 128 SV streams, an upper bound that we previously determined to be of interest for 1 Gbit/s Ethernet. For the Shared-Nothing variant, the question is whether the packet processing for all streams, including deduplication, encapsulation and forwarding, has a cost that is lower than 250 μ s. For the Host-Cache variants, the preconditioning operations must also fit in the 250 μ s computation window. Second, the Shared-Nothing variant is built on the interrupt-driven model. Therefore, for each CPU that is partly used by the network stack, the remaining time can be given to other host processes.

Methodology

We start a VM that services two SV streams. The no-monitor VPAC responds with a trip after processing both. We gather the latencies and use these as a baseline. We then increase the number of SV streams that are processed by the host-side part of the network stacks, without increasing the number of SV streams that are delivered to the VM. To achieve that for the Shared-Nothing variant, we use another tap device as a drain. The two SV streams that are processed by the VM are always sent last, the first stream on LAN A, the second stream on LAN B. This ensures that the VPAC in the VM can only answer with a trip after all other streams have been processed host-side. We then obtain the average cost of host-side processing per 250 μ s and per core by subtracting the average of the baseline latencies. The load generator sends the burst of SV streams every 250 μ s. We gather 100 000 samples per stream count.

For the Shared-Nothing variant, we run the experiment with one IRQ configured per physical interface, one time with both being pinned to the same core, one time with separate core affinities. For the Host-Cache variants, we quantify the time that remains for preconditioning in terms of slack. Therefore, the preconditioning in this case only copies some dummy values into IVSHMEM.

To better understand the costs induced by the Linux networking stack, we also measure a setup that we have not proposed as a separate solution. We already have an eBPF implementation of PRP. We can attach it to the Linux stack, this time allowing receive-traffic to flow through the bridge, rather than intercepting it early with an AF_XDP application. In the results depicted in Figure 6.3, we named this setup “XDP

plain". As with the Shared-Nothing variant, we configured one IRQ for each of the two physical interfaces, and we measured the setup with both IRQs pinned to one core, and with both pinned to separate cores.

Results

Because the mean and the 99th percentile were close in all the measurements, we omit them from the plot for readability.

For small numbers of SV streams, all variants and the added setup perform similarly. There are considerable differences starting at 64 SV streams: the DPDK host-cache needs about 16 μ s of computation, whereas the Shared-Nothing variant consumes 89 μ s on one core, or 72 μ s per core in the two-core setup. At 128 streams, a process that is scheduled on the same core as the Shared-Nothing packet processing receives less than a quarter of the window.

The differences between the two host-cache implementations are small, which validates the busy-poll AF_XDP architecture. At 128 streams, the DPDK host-cache needs 31 μ s and the AF_XDP host-cache 35 μ s. In both cases, the slack suffices for all preconditioning operations.

The XDP plain setup performs considerably better than our Shared-Nothing variant. The difference in cost can be explained by two factors. First, the XDP implementation of PRP drops duplicates before `sk_buff` allocation. Second, our implementation of the deduplication algorithm needs fewer memory allocations and computes less, given critical traffic.

These results also motivate our recommendation of pinning both IRQs to the same core. If asymmetric delay is rare, the first frame to arrive and its duplicate reach the physical interfaces at approximately the same time. The instance of the PRP kernel module that starts processing the stream slightly later has to waste cycles until the other instance is finished, because of the per-node spinlock discussed in Subsection 4.2.1. That is the reason why per-core CPU usage is not halved in the two-core setup. We have not observed more jitter in any of the IRQ configurations, and the differences in latency are small. Toeplitz hashes usually do not include L2 headers, so equal spread is not guaranteed. Therefore, we recommend avoiding RSS.

Recommendation #3: Receive queue and IRQ affinity

Configure one receive queue per physical interface and pin both IRQs to the same core, unless the NIC supports n-tuple filtering on destination MAC addresses.

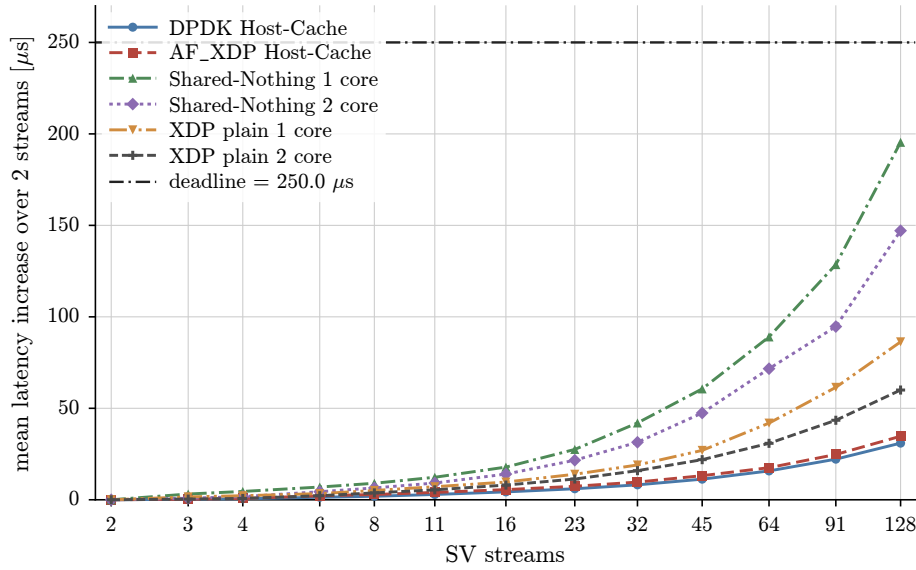


Figure 6.3: Average CPU Usage for each core engaged in packet processing per 250 μ s.

6.4 Round-Trip Time

Research Question

RQ 2 *How do the three stacks compare in mean round-trip time and in tail latency, and which layer drives the outliers?*

As discussed in Section 3.4, the reaction time of the VPAC directly corresponds to a sum of E2E latencies. A round-trip time measurement from the load generator to the VPAC allows us to examine the empirical distribution of the response times. Not only the average, but also the high percentiles and the variation in latencies are important.

Methodology

The load generator sends one SV stream on both LANs every 250 μ s. After the stream reaches the VPAC, it responds with a trip. Both the preconditioning and the protection functions are NOPs, our main interest being the overhead introduced by the network stacks. We gather 10 000 000 samples.

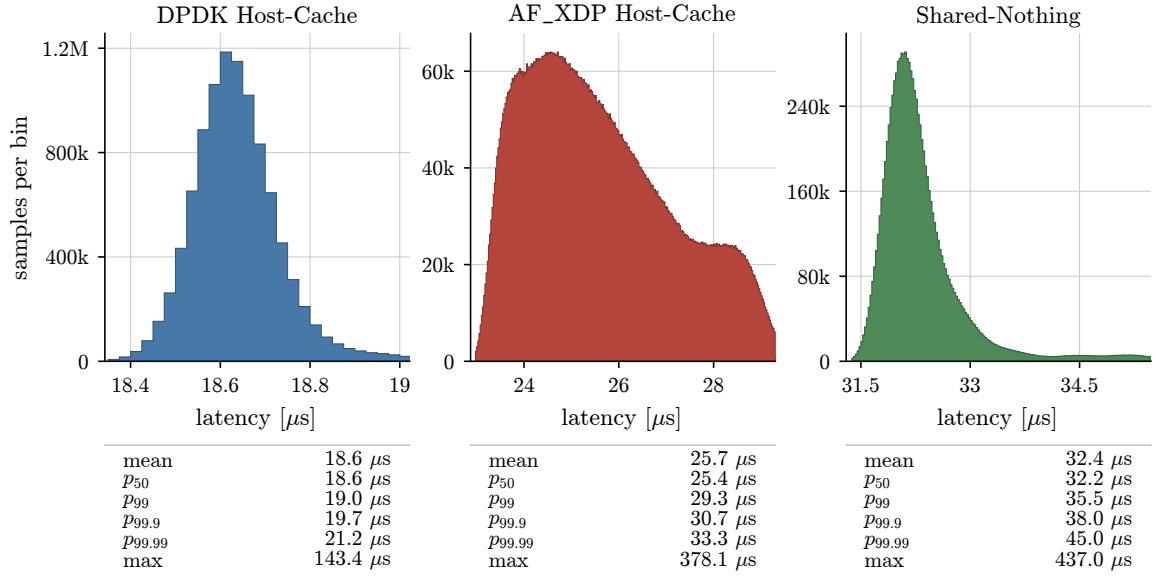


Figure 6.4: Histograms of the round-trip times up to the 99th percentile, with relevant statistics including the deep tail below.

Results

We have displayed the results in Figure 6.4. Compared to the Shared-Nothing variant, the DPDK variant has an RTT mean that is smaller by 43%, and a difference between the 99.99th percentile and the 50th percentile that is smaller by 79%. The AF_XDP variant represents the performance middle ground, with an RTT mean that is smaller by 21% and a difference between the 99.99th percentile and the 50th percentile that is smaller by 38%. The fact that the DPDK variant succeeds in reducing the number of outliers and their magnitude suggests that the tails of the other distributions are mostly driven by the vhost-net thread. The VM must exit to notify it when transmitting packets, and the wake-up cost host-side contributes to the tail.

The maximum of the DPDK variant is driven by virtualisation overhead. As we see in Subsection 6.5.2, the variance does not increase substantially when receiving or transmitting more, which points to a constant cost that does not scale with load.

6.5 Model Validation

6.5.1 Host-Cache VM Capacity

Research Question

RQ 3 *Does the capacity condition of Equation 6.3 predict protection vCPU usage with sufficient accuracy to derive the per-window protection budget for a given stream count?*

We restate the host-cache VM capacity condition

$$M \cdot (t_{\text{handoff}} + t_{\text{protection}}) + t_{\text{send}} < T \quad (6.10)$$

We measured the cost of the IPC protocol in Subsection 6.1.5. We validate our approach experimentally.

If our capacity condition accurately captures protection vCPU usage, then we can express the time that remains per-window for protection computations for a given SV stream number.

Methodology

We time the duration of the processing in the VM in the DPDK variant, with protection as NOP, for both the no-monitor setup and the monitor setup. In the no-monitor setup there are only the ring buffer scans without the IPC. In the monitor setup, the mean should increase by roughly 3.26 μs per stream.

To measure the effect that the number of VPACs has, we also measure the average cost at 64 SV streams with 1–8 VPACs, while equally splitting streams between them.

Results

Figure 6.5 displays the results. At 64 SV streams, the monitor setup has 222 μs of cost from management alone. The remaining 28 μs are insufficient for protection. Beyond 64 SV streams, packet loss occurs because the capacity condition is breached. If the per-stream protection cost is in the low single-digit microseconds, the maximum number of SV streams that can be serviced by one protection vCPU is between 40 and 55 streams.

Our prediction has the highest error of 1.7 μs at 32 streams. We attribute the error to the fact that in the monitor setup the VPAC also has to access the ring buffer once per stream.

Figure 6.6 shows the effect of multiple VPACs on average cost at 64 SV streams. Up to three VPACs, the difference from the one-VPAC mean is not significant. Starting with four VPACs, the delta varies between 4 and 6.6 μs . These differences are caused by

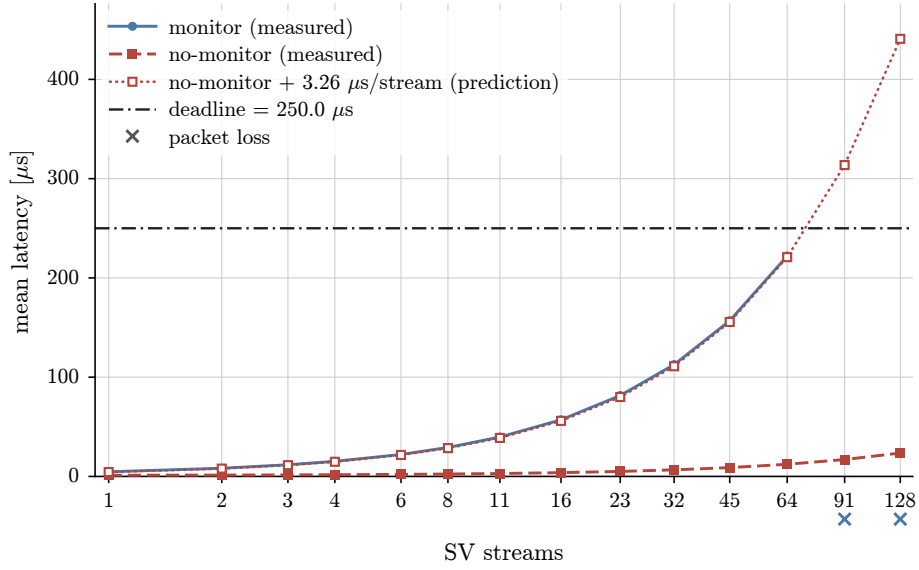


Figure 6.5: Management cost at different SV stream numbers for both the monitor and the no-monitor setups and the prediction of the monitor setup cost from the no-monitor setup cost. $3.26 \mu\text{s}$ represents the average cost of the IPC protocol.

the indirect cost of context switches. This cost can be absorbed into the scan variable. Because the differences are small, we conclude that the VPAC count is not a parameter with a meaningful impact in the observed range of values.

6.5.2 Worst-Case Burst

Research Question

RQ 4 *Does the makespan derivation for the DPDK variant predict end-to-end latency under an accumulated burst of five SV windows?*

In Subsection 6.1.4, we developed a model that predicts latencies given worst-case costs of packet-processing and protection operations. We verify whether the theoretical schema matches the real behaviour of the system.

Methodology

Worst-case execution time analysis on Linux and commercial server hardware is difficult. Therefore, we relax the constraints to build a validation procedure. We return to our

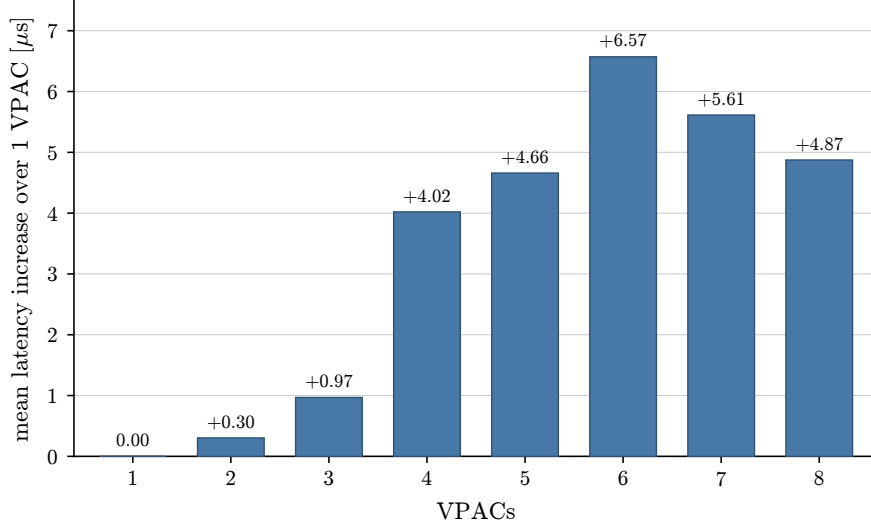


Figure 6.6: Difference in average management cost at 64 SV streams for different VPAC numbers, using the one-VPAC measurement as a reference.

derived latency formula for one VM

$$\text{Latency} = t_{\text{SV-accept}} + (K + 1) \cdot L_V - t_{\text{context-switch}} + t_{\text{GOOSE-PRP}} + 2 \cdot D. \quad (6.11)$$

As K increases, our formula predicts that the latency increases by the costs of the VM tasks. We write B for the random variable denoting the baseline, with $K = 0$. We obtain its distribution empirically. We then write $\text{IPC}_{i,j}$ for the random variable denoting the j -th IPC protocol of the i -th window contained in the burst. We use our estimation from Subsection 6.1.5 and assume all the IPC protocol random variables of the burst to be independent and identically distributed. Analogously, we write Send_i for the random variable denoting the i -th send of the burst, with symmetric considerations. Assuming everything else to be constant and expanding L_V , we obtain

$$L_k = B + \sum_{i=1}^k \sum_{j=1}^M \text{IPC}_{i,j} + \sum_{i=1}^k \text{Send}_i + k \cdot M \cdot (t_{\text{scan}} + t_{\text{protection}}) \quad (6.12)$$

where L_k is the random variable denoting the worst-case latency for $K = k$.

Because we have assumed all of the random variables to be independent, if we obtain the distributions of the random variables empirically, we can estimate the distribution of L_k by using convolutions. We now have a validation procedure: we pick values for the constants, we measure the baseline latencies and the latencies for an increasing number of windows, and then we compare our prediction to the measured distribution.

We choose the constants so that an incorrect makespan derivation would be visible in the results. We set $M = 4$, $t_{\text{protection}} = 4 \mu\text{s}$ and $t_{\text{precondition}} = 2 \mu\text{s}$.

Results

The experiment is depicted in Figure 6.7. The very last send of the burst is taken into consideration for the worst-case latency. The constant model is a pure location shift of the baseline by $k \cdot \text{STEP}$ with

$$\text{STEP} = \text{Median} + 4 \cdot \mathbb{E}[\text{IPC}] + \mathbb{E}[\text{Send}] = 16092 \text{ ns} + 13019 \text{ ns} + 1018 \text{ ns} = 30129 \text{ ns} \quad (6.13)$$

where Median is obtained by fitting a linear model over the five windows. It represents the effect of $M \cdot (t_{\text{protection}} + t_{\text{scan}})$. When shifting the IPC protocol distribution to match the true average (see Subsection 6.1.5), we reach a mean of 3255 ns rather than 3260 ns because of approximation errors. As previously described, the full model replaces the expectation values by the random variables. While the location shift model is close to the measured distribution, it deviates by up to $1.8 \mu\text{s}$ at the 99.99th percentile. It also explains less of the variance, because it can only contain the variance of the baseline. For $K = 4$, the full model explains 61% of the variance, the constant model 29%. We attribute the remaining unexplained variance to the way the protection computation is simulated.

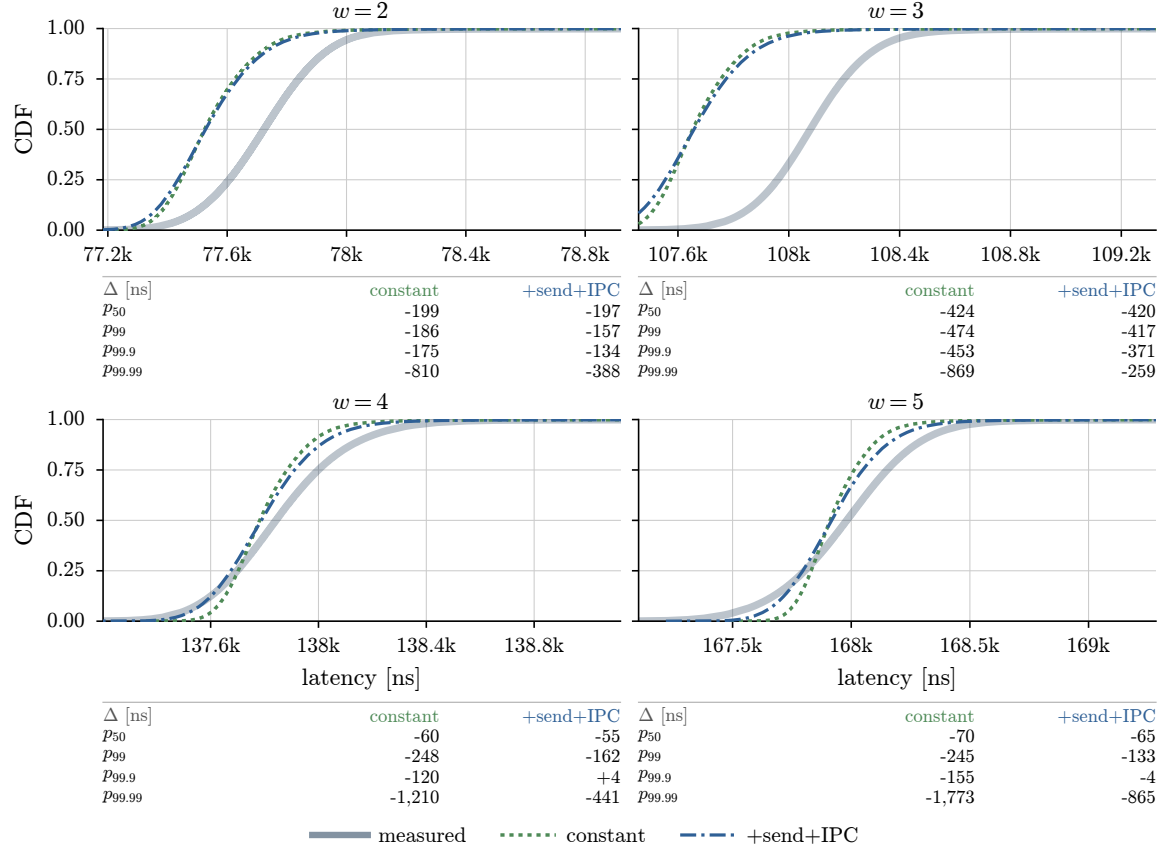


Figure 6.7: Measured worst-case burst latencies compared against the predictions. The x-axes of the CDF plots are clipped to the 0.05th percentile to the left, and to the 99.9th percentile to the right. The table below each plot depicts the absolute error of the predictions in nanoseconds. $W = K + 1$ represents the size of the burst in windows of computation.

7 Related Work

7.1 Containerised VPACs

Our approach aims at maximum isolation assuming no SR-IOV capability. There has been research on a more relaxed approach, that of containerising VPACs. The core of the containerised solution of Schönborn, Birke, Kozhaya, and Sivanthi [73] is a Docker image that encapsulates all dependencies, support entities and an application that can handle the protection for 20 SV streams. They achieved their goal of sub-1 ms maximum application timings by giving the application four isolated CPUs. The measurement was done under both simulated non-RT and RT load. The authors determined that poor resource isolation and virtual network delays, rather than scheduling, led to timing failures. While virtual networking alternatives are discussed, no further details are given about the internals of the virtualised protection and control application.

7.2 Industry White Papers

The topic of this thesis stems from the protection industry. Therefore, we review industry white papers carefully. We discuss an ABB white paper [2], which describes a real-life substation pilot experiment that was realised jointly with Finnish DSO Caruna, and an Intel white paper [36], which compares multiple approaches to virtualised protection and control.

The ABB white paper is an industry rendition of the setup that we described in Section 7.1. It seeks to validate the containerised approach to virtualised protection and control by testing it continuously for a long period of time in a 110 kV/20 kV substation in western Finland. While a bare-metal CPC solution was running as the main protection system, a separate virtualised solution was used in standby mode, executing the same protection functions as the CPC device, but without the possibility of sending GOOSE trips.

The substation had 13 measurement points. They used macvlan for their virtual networks and Linux patched with PREEMPT_RT on the host. They deployed two virtualised protection and control containers and, for over a year, both matched the behaviour of the CPC device. This is the experiment that led to ABB's commercial virtualised

protection and control product that was released at the beginning of 2023 [3].

Out of the two, the Intel white paper is more relevant to us, because it engages more with the internals of VPAC solutions. It contains an architectural discussion that forms the basis for our distinction between the Shared-Nothing variant and the Host-Cache variants. The first approach is to directly port discrete IEDs, as VMs, onto a host, each having its own complete ensemble of capabilities. The second is based on shared functionality. The latter is more efficient given higher SV stream counts, but requires an open API to communicate with shared modules. The white paper also presents Kalkitech’s virtualised protection and control solution and its evaluation, but, because of their down-sampling technique that changes cycle times, the results are difficult to compare with ours.

7.3 DPDK-based Low-Latency Applications

DPDK’s PMD implementations are mainly geared towards reducing the processing cost per packet [64]. There is a vast literature documenting DPDK’s throughput performance, but the literature on its viability for low-latency applications is scattered across industrial automation and control, high-reliability network functions and scientific computing.

In the particle acceleration field, the Elettra synchrotron group studied the effects of DPDK-powered Ethernet for their most jitter-sensitive real-time application, the fast orbit feedback system [27]. The use case corresponds to a feedback response loop with a frequency of 10 kHz. The server is connected to four 10 GbE ports. Every 100 μ s, 24 UDP packets of 80 bytes each, representing the inputs, are received through each of the ports. They used the mainline Linux kernel for their evaluation, first paired with POSIX sockets, and then with DPDK. None of the tests with POSIX sockets reported less than 0.8 ms of jitter in the worst case, while DPDK achieved sub-40 μ s jitter.

UDPDK is a DPDK-based middleware for low-latency NFs [46]. The work is motivated by the critical latency requirements imposed by various 5G use cases. The authors build on top of the Environment Abstraction Layer to provide a POSIX-like API for UDP communication, with the purpose of requiring as little modification as possible from the applications above the interface. UDPDK applications are multi-process applications, with one dedicated polling process and one or more application processes. To communicate with the polling process, the application processes open UDPDK sockets, which are implemented using message queues. When the authors used UDPDK to accelerate the OpenAirInterface, an open-source software stack for packet-processing in RAN scenarios, the average latency decreased from 95 μ s to 29 μ s, the POSIX API being the baseline. In our use case, we achieve a relative reduction in average latency

of the same order. The authors mention that this performance increase comes at the cost of one fully busy CPU, unavoidable because of the need for a polling process.

Gallenmüller, Naab, Adam, and Carle [28] also use DPDK for reducing latencies of 5G-related NFs. The authors are interested in measuring the latency performance of a security NF based on Snort 3. The target latency bound is 0.5 ms. First, they measure the latency in a typical setting: a Debian buster with KVM virtualises NFs of different network slices. Even at a packet rate of 10 kpps, the 99.99th percentile already violates the delay requirements. After exhaustively listing the possible causes, they derive guidelines for hosting low-latency NFs. We have applied or discussed most of them: building on specialised packet processing frameworks like DPDK, attaching VMs with SR-IOV, disabling energy-saving mechanisms, isolating cores. The remaining two guidelines, applying Intel CAT and switching to an RT kernel, belong to future work. In particular, cache behaviour and the risk of host-side cache pollution are important points that we have not covered.

The authors first measure the packet processing overhead of DPDK, and then the NF's performance by using a DPDK-based version of Snort. At low and moderate packet rates, latency stays below 125 μ s. Like us, the authors also suggest high utilisation to limit the additional energy cost induced by polling and CPU tuning.

7.4 Kernel Bypassing for TSN

TSN is sometimes used in industrial automation scenarios with cycle times of less than 50 μ s [80]. An engineer designing a TSN end station for such use cases has two options: an implementation on a dedicated hardware platform or a software implementation based on the Linux kernel. Within the software category, solutions that shorten or bypass the kernel networking stack are popular because they can match the latency and throughput of hardware solutions while retaining flexibility.

Fejes, Orosi, Varga, and Farkas [26] implemented Frame Replication and Elimination for Reliability (FRER) in XDP. FRER is a TSN standard that offers a redundancy mechanism by replicating each packet via multiple paths [24]. The main difference between FRER and PRP is that PRP provides replication and elimination only at end nodes. The authors implemented both functions in XDP, and compared the performance with that of a user-space implementation. The evaluation reveals that XDP FRER performs better than user-space FRER, and that it is unaffected by CPU load. This is the expected behaviour because XDP runs in the high-priority softIRQ context, while the user-space NF is a regular process that can be arbitrarily preempted by the kernel. With similar goals, Kobori, C. Lee, and Hirotsu [44] implemented time synchronisation in eBPF.

Garbugli, Rosa, Bujari, and Foschini [29] designed KuberneTSN, an architecture for a DPDK-accelerated container overlay network that handles TSN scheduling. Bare-metal implementations of TSN scheduling use the `SO_TXTIME` socket option, which allows the application to specify the time of transmission. Because the transmission time is local to the container network namespace and cannot reach the host network, the authors created a Multus-based plugin that implements the packet scheduler. They used DPDK to forward the packets over the virtual L2 link, and OVS-DPDK for further forwarding to the physical interface. In terms of E2E latency and jitter, the architecture matches a bare-metal implementation.

8 Summary and Conclusion

The thesis presents three network stacks for VPACs and evaluates them from within a framework defined by end-to-end latency, determinism, interoperability and resource usage. We propose a Shared-Nothing variant that rests on the standard Linux networking stack and two Host-Cache variants, one based on XDP, and one that makes use of the DPDK framework. The Shared-Nothing variant is designed to provide VPACs with the unchanged UNIX socket API. The two Host-Cache variants leverage kernel bypassing for lower packet-processing costs at increased SV stream counts and replace the interrupt-driven model with the poll-based model for predictability. We implement network redundancy and forwarding in software and show that the QoS enforced by the IEC 61850 standard can be met without specialised hardware.

At low SV stream numbers, the Shared-Nothing stack is more resource-efficient because the packet-processing core leaves space for other processes to be scheduled. At 128 SV streams, both Host-Cache variants consume at least $150\mu\text{s}$ less than the Shared-Nothing variant, given the recommended IRQ configuration. The rest of the per-window time is used for the preconditioning operation, mitigating the cost of polling.

We find that all proposed network stacks achieve latencies under $500\mu\text{s}$. The DPDK variant produces latencies with the lowest mean, the lowest jitter and a maximum observed latency over 40 minutes of measurement that is lower by roughly a factor of three. The AF_XDP variant replaces the standard Linux networking receive path with a low-latency alternative, but this replacement is not sufficient to reduce maximum latencies by a significant amount. This indicates that the vhost-net thread and its scheduling patterns cause stalls in virtual network forwarding on transmit.

Bypassing the complexity of the Linux kernel networking stack with DPDK simplifies the task of creating a theoretical model of the system. Given some assumptions about the relative costs of packet-processing and protection operations, the makespan of the system is derived under network burst conditions. Furthermore, the low jitter of PMD calls enables the probabilistic validation of the model. However, these are measured latencies, and not static worst-case execution time bounds.

Based on the evidence, we recommend the Shared-Nothing variant for small and medium-sized substations. Beyond roughly 64 SV streams, interoperability is the only metric that disadvantages the Host-Cache variants. We summarise our findings in the

following recommendation:

Recommendation #4: Network stack choice

Shared-Nothing	For small and medium-sized substations, because it does not require a different receive API.
DPDK Host-Cache	For large substations, when prioritising bounded latency.
AF_XDP Host-Cache	For large substations, when prioritising ease of integration into existing deployments.

The source code can be found at <https://github.com/radurac/vpac-bsc-thesis-artifact>.

9 Future Work

RT Scheduling

We discussed best-effort traffic from a networking perspective in Section 5.4, but a further aspect of non-RT tasks remains unresolved. In both the Shared-Nothing and the AF_XDP variants, the vhost-net thread is on the critical traffic’s hot path. For our measurements, we pinned the QEMU emulator (including the vhost-net thread) to an isolated core with no other processes. If non-RT tasks are to be scheduled on the same core, PREEMPT_RT host-side is necessary. By construction, the Host-Cache variants cannot accommodate non-RT tasks on the protection vCPUs. With the caveats in Section 4.1, the Shared-Nothing variant can allow for such tasks. In that case, PREEMPT_RT becomes a necessity in the VM. Our measurements were based on the stock kernel, so any such changes need to be measured carefully in future work.

We have not investigated the impact of host-side non-RT tasks on cache performance. While the host-caches effectively own the L1 cache, because they are sole applications on isolated cores, the Shared-Nothing variant allows for other applications to be co-located on the cores where packet processing runs. The PRP kernel module does have important data structures to store, as discussed in Subsection 4.2.1, so the effects that co-location has on caching should be measured.

Worst-Case Cost Estimation

Although we did derive a latency decomposition model for the DPDK variant, we did not measure any worst-case cost terms. Our validation adapts the decomposition and models certain critical variables as random variables, but nowhere in the thesis do we attempt to resolve worst-case costs.

Rather than a pure worst-case execution time derivation, future work can apply a method such as that of Iyer, Pedrosa, Zaostrovnykh, et al. [38]. The paper proposes the notion of a performance contract, formulated in terms of performance critical variables, for predicting NF performance. The advantage of such an approach is that it strikes a balance between accuracy and human legibility. Performance-critical variables here include whether a frame is dropped because it is a duplicate, or whether a frame contains information that would trigger a trip. Bolt, the tool for computing performance contracts, includes DPDK calls in its analysis.

Isolation of Co-located VPACs

Perfect or near-perfect networking isolation can only be achieved with an SR-IOV-capable NIC (see Section 4.4), but there are available options to at least increase the degree of isolation between co-located VPACs. The modified VPAC is already controlled by the monitor when receiving data, and our implementation of the latter can be extended to provide the necessary level of isolation. However, the VPACs still use the same device to transmit packets. An eBPF-based filter could be developed in the future to drop the packets of a malfunctioning VPAC to prevent it from needlessly overloading the network stack. The main task would then be to correctly identify what constitutes normal and abnormal behaviour.

Shared-Nothing VM Capacity

We have not estimated the average packet-processing cost in the Shared-Nothing VM. The difficulty comes from the involvement of the virtualisation stack and from the fact that softIRQs do not run in a dedicated context. NAPI threading can be enabled for better observability, but it introduces context-switch-related costs. Emmerich, Raumer, Wohlfart, and Carle [23] have measured the cost of packet processing with different networking setups in the host. Future work can repeat the measurement in the VM, also taking into consideration virtualisation-associated penalties.

Abbreviations

PRP Parallel Redundancy Protocol

DPDK Data Plane Development Kit

XDP eXpress Data Path

SV Sampled Values

GOOSE Generic Object-Oriented Substation Events

MU Merging Unit

IED Intelligent Electronic Device

CPC Centralised Protection and Control

NIC Network Interface Card

HSR High-availability Seamless Redundancy

DANP Doubly Attached Node implementing PRP

SAN Singly Attached Node

VM Virtual Machine

QoS Quality of Service

VPAC Virtualised Protection and Control

PMD Poll Mode Driver

IPC Inter-Process Communication

IVSHMEM Inter-VM Shared Memory

SR-IOV Single Root I/O Virtualization

TSN Time-Sensitive Networking

NAPI New API

CB Circuit Breaker

RedBox Redundancy Box

RCT Redundancy Control Trailer

RSS Receive Side Scaling

LRE Link Redundancy Entity

PF Physical Function

VF Virtual Function

OVS Open vSwitch

SPSC Single-Producer/Single-Consumer

FRER Frame Replication and Elimination for Reliability

List of Figures

2.1	The conventional architecture.	5
2.2	The CPC architecture.	6
2.3	The redundancy control trailer as defined in the standard. It is attached at the end of the payload.	7
2.4	Example of a PRP-compliant network topology. DANPs can communicate with all other nodes. SANs can communicate with DANPs, or SANs on the same LAN. A RedBox is a hardware device that allows a SAN to act as a DANP. The precise layout of the LANs does not have to be the same. The figure is a simplification of one in [35].	8
2.5	The structure of a generic software solution for VPACs.	12
3.1	Overview of the three stack variants. The green arrows mark the receive path and the purple arrows mark the transmit path. The circled numbers match with the steps of the workflows.	16
4.1	In-depth design of the three stack variants: Shared-Nothing (left), AF_XDP-based Host-Cache (middle), DPDK-based Host-Cache (right). The NIC box stands in for both ports.	24
4.2	IPC protocol diagram.	26
5.1	The TC program is attached to one of the interfaces, and only that one interface is enslaved to the bridge.	34
6.1	Histograms of the costs up to the 99th percentile, with relevant statistics including the deep tail below.	48
6.2	A diagram of the test bench, depicting the load generation machine, the device under test, and the PRP setup.	50
6.3	Average CPU Usage for each core engaged in packet processing per 250 μ s.	53
6.4	Histograms of the round-trip times up to the 99th percentile, with relevant statistics including the deep tail below.	54

6.5	Management cost at different SV stream numbers for both the monitor and the no-monitor setups and the prediction of the monitor setup cost from the no-monitor setup cost. $3.26\ \mu\text{s}$ represents the average cost of the IPC protocol.	56
6.6	Difference in average management cost at 64 SV streams for different VPAC numbers, using the one-VPAC measurement as a reference. . . .	57
6.7	Measured worst-case burst latencies compared against the predictions. The x-axes of the CDF plots are clipped to the 0.05th percentile to the left, and to the 99.9th percentile to the right. The table below each plot depicts the absolute error of the predictions in nanoseconds. $W = K + 1$ represents the size of the burst in windows of computation.	59

List of Tables

6.1	Notation used in the capacity and latency models. Some other notation is introduced when needed in the relevant section.	44
-----	--	----

Bibliography

- [1] *A journey to the vhost-users realm*. Red Hat. Sept. 24, 2019. URL: <https://www.redhat.com/en/blog/journey-vhost-users-realm> (visited on 08/07/2026).
- [2] ABB. *Virtualization of protection and control – Evaluation and deployment considerations*. 2023. URL: https://library.e.abb.com/public/3f5ff4e1c22b49a28987ffcac74eae96/White%20paper%20NGA001642_RevA_EN_finalb.pdf (visited on 07/27/2026).
- [3] *ABB launches the world's first virtualized protection and control solution*. ABB. Jan. 18, 2023. URL: <https://new.abb.com/news/detail/98682/abb-launches-the-worlds-first-virtualized-protection-and-control-solution> (visited on 08/18/2026).
- [4] *AF_XDP*. ebpf.io. URL: https://docs.ebpf.io/linux/concepts/af_xdp/ (visited on 08/02/2026).
- [5] *ANSI protective functions*. Typhoon HIL. 2026. URL: https://www.typhoon-hil.com/documentation/typhoon-hil-software-manual/References/ansi_protective_functions.html (visited on 08/05/2026).
- [6] J. Araujo, J. Lázaro, A. Astarloa, A. Zuloaga, and N. Moreira. "Duplicate and circulating frames discard methods for PRP and HSR (IEC62439-3)." In: *IECON 2013 - 39th Annual Conference of the IEEE Industrial Electronics Society*. 2013, pp. 4451–4456. doi: 10.1109/IECON.2013.6699852.
- [7] T. Barbette, G. P. Katsikas, G. Q. Maguire, and D. Kostić. "RSS++: load and state-aware receive side scaling." In: *Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies*. CoNEXT '19. Orlando, Florida: Association for Computing Machinery, 2019, pp. 318–333. ISBN: 9781450369985. doi: 10.1145/3359989.3365412.
- [8] A. Beifuß, D. Raumer, P. Emmerich, T. M. Runge, F. Wohlfart, B. E. Wolfinger, and G. Carle. "A study of networking software induced latency." In: *2015 International Conference and Workshops on Networked Systems (NetSys)*. 2015, pp. 1–8. doi: 10.1109/NetSys.2015.7089065.
- [9] A. Belkhiri, M. Pepin, M. Bly, and M. Dagenais. "Performance analysis of DPDK-based applications through tracing." In: *J. Parallel Distrib. Comput.* 173.C (Mar. 2023), pp. 1–19. ISSN: 0743-7315. doi: 10.1016/j.jpdc.2022.10.012.

- [10] S. Bhat. *Accelerating VPR and VPAC tools development with Kalkitech's IEC61850 based Framework*. Kalkitech. Feb. 17, 2025. URL: <https://kalkitech.com/iec-61850-development-frameworks/> (visited on 07/29/2026).
- [11] W. de Bruijn. *Add napi for virtio-net transmit completion processing*. linux-netdev mailing list. Apr. 24, 2017. URL: <https://marc.info/?l=linux-netdev&m=149305618416472> (visited on 08/13/2026).
- [12] R. Chen and G. Sun. "A Survey of Kernel-Bypass Techniques in Network Stack." In: *Proceedings of the 2018 2nd International Conference on Computer Science and Artificial Intelligence*. CSAI '18. Shenzhen, China: Association for Computing Machinery, 2018, pp. 474–477. ISBN: 9781450366069. DOI: 10.1145/3297156.3297242.
- [13] *Concurrency*. ebpf.io. June 7, 2025. URL: <https://docs.ebpf.io/linux/concepts/concurrency/> (visited on 08/12/2026).
- [14] *Configuring PRP*. Cisco. 2023. URL: https://www.cisco.com/c/dam/en/us/td/docs/switches/lan/industrial/software/configuration/guide/b_prp_ie4k_5k.html (visited on 08/06/2026).
- [15] J. Corbet. *Accelerating networking with AF_XDP*. Apr. 9, 2018. URL: <https://lwn.net/Articles/750845/> (visited on 08/05/2026).
- [16] J. Corbet. *Attaching eBPF programs to sockets*. Dec. 10, 2014. URL: <https://lwn.net/Articles/625224/> (visited on 08/15/2026).
- [17] J. Corbet. *Per-vector software-interrupt masking*. Feb. 15, 2019. URL: <https://lwn.net/Articles/779738/> (visited on 08/02/2026).
- [18] J. Corbet. *Smarter IRQ suspension in the networking stack*. Feb. 11, 2025. URL: <https://lwn.net/Articles/1008399/> (visited on 08/17/2026).
- [19] E. Csanyi. *Protection Relay - ANSI Standards*. Electrical Engineering Portal. 2025-01-27. URL: <https://electrical-engineering-portal.com/protection-relay-ansi-standards> (visited on 08/05/2026).
- [20] J. Damato. *Monitoring and Tuning the Linux Networking Stack: Receiving Data*. packagecloud blog. June 21, 2016. URL: <https://blog.packagecloud.io/monitoring-tuning-linux-networking-stack-receiving-data/> (visited on 07/30/2026).
- [21] Y. Dong, X. Yang, J. Li, G. Liao, K. Tian, and H. Guan. "High performance network virtualization with SR-IOV." In: *Journal of Parallel and Distributed Computing* 72.11 (2012). Communication Architectures for Scalable Systems, pp. 1471–1480. ISSN: 0743-7315. DOI: <https://doi.org/10.1016/j.jpdc.2012.01.020>.

- [22] P. Emmerich, D. Raumer, A. Beifuß, L. Erlacher, F. Wohlfart, T. M. Runge, S. Gallenmüller, and G. Carle. “Optimizing latency and CPU load in packet processing systems.” In: *2015 International Symposium on Performance Evaluation of Computer and Telecommunication Systems (SPECTS)*. 2015, pp. 1–8. DOI: 10.1109/SPECTS.2015.7285275.
- [23] P. Emmerich, D. Raumer, F. Wohlfart, and G. Carle. “Assessing Soft- and Hardware Bottlenecks in PC-based Packet Forwarding Systems.” In: 2015.
- [24] D. Ergenç and M. Fischer. “On the Reliability of IEEE 802.1CB FRER.” In: *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*. 2021, pp. 1–10. DOI: 10.1109/INFOCOM42981.2021.9488750.
- [25] V. Fang, T. Lévai, S. Han, S. Ratnasamy, B. Raghavan, and J. Sherry. *Evaluating Software Switches: Hard or Hopeless?* Tech. rep. UCB/EECS-2018-136. Oct. 2018.
- [26] F. Fejes, F. Orosi, B. Varga, and J. Farkas. *Lightweight Implementation of Per-packet Service Protection in eBPF/XDP*. 2023. arXiv: 2312.07152 [cs.NI].
- [27] G. Gaio and G. Scalamera. “Development of Ethernet Based Real-Time Applications in Linux Using DPDK.” In: *17th International Conference on Accelerator and Large Experimental Physics Control Systems*. 2020, MOPHA044. DOI: 10.18429/JACoW-ICALEPCS2019-MOPHA044.
- [28] S. Gallenmüller, J. Naab, I. Adam, and G. Carle. “5G URLLC: A Case Study on Low-Latency Intrusion Prevention.” In: *IEEE Communications Magazine* 58.10 (2020), pp. 35–41. DOI: 10.1109/MCOM.001.2000467.
- [29] A. Garbugli, L. Rosa, A. Bujari, and L. Foschini. “KuberneTSN: a Deterministic Overlay Network for Time-Sensitive Containerized Environments.” In: *ICC 2023 - IEEE International Conference on Communications*. 2023, pp. 1494–1499. DOI: 10.1109/ICC45041.2023.10279214.
- [30] L. Hangbin. *An introduction to Linux bridging commands and features*. Red Hat. Apr. 6, 2022. URL: <https://developers.redhat.com/articles/2022/04/06/introduction-linux-bridging-commands-and-features#multicast> (visited on 08/04/2026).
- [31] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller. “The eXpress data path: fast programmable packet processing in the operating system kernel.” In: *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies*. CoNEXT ’18. Heraklion, Greece: Association for Computing Machinery, 2018, pp. 54–66. ISBN: 9781450360807. DOI: 10.1145/3281411.3281443.

- [32] *HowTo Guides - 9. Virtio_user as Exception Path*. DPDK Project. URL: https://doc.dpdk.org/guides/howto/virtio_user_as_exception_path.html (visited on 08/13/2026).
- [33] *IEC 61850 Sampled Values protocol*. Typhoon HIL. 2026. URL: https://www.typhoon-hil.com/documentation/typhoon-hil-software-manual/References/iec_61850_sampled_values_protocol.html (visited on 07/27/2026).
- [34] “IEEE Guide for Centralized Protection and Control (CPC) Systems within a Substation.” In: *IEEE Std C37.300-2024* (2025), pp. 1–130. DOI: 10.1109/IEEESTD.2025.10994391.
- [35] *Industrial communication networks - High availability automation networks - Part 3: Parallel Redundancy Protocol (PRP) and High-availability Seamless Redundancy (HSR)*. International Electrotechnical Commission, 2026.
- [36] Intel. *Virtual Protection Relay White Paper*. 2024. URL: <https://builders.intel.com/docs/networkbuilders/virtual-protection-relay-white-paper-1709299398.pdf> (visited on 07/27/2026).
- [37] *Inter-VM Shared Memory device*. QEMU Project. 2026. URL: <https://www.qemu.org/docs/master/system/devices/ivshmem.html> (visited on 08/05/2026).
- [38] R. Iyer, L. Pedrosa, A. Zaostrovnykh, S. Pirelli, K. Argyraki, and G. Candea. “Performance Contracts for Software Network Functions.” In: *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. Boston, MA: USENIX Association, Feb. 2019, pp. 517–530. ISBN: 978-1-931971-49-2.
- [39] M. Karlsson and B. Töpel. “The Path to DPDK Speeds for AF_XDP.” In: *Linux Plumbers Conference (LPC)*. Vancouver, BC, Canada, 2018.
- [40] M. Karlsson and B. Töpel. *The Path to DPDK Speeds for AF_XDP - Presentation*. 2018. URL: https://lpc.events/event/2/contributions/99/attachments/98/116/lpc18_pres_af_xdp_perf-v3.pdf (visited on 08/05/2026).
- [41] J. Karrenpalo. *net: hsr: Fix PRP duplicate detection*. Linux kernel Git repository. Mar. 7, 2025. URL: <https://git.kernel.org/pub/scm/linux/kernel/git/netdev/net-next.git/commit/?id=05fd00e5e7b1> (visited on 08/12/2026).
- [42] H. Kim, J. Lee, S. Hong, and C. Han. “Secure IVSHMEM: End-to-End Shared-Memory Protocol with Hypervisor-CA Handshake and In-Kernel Access Control.” In: *Proceedings of the International Symposium on Memory Systems*. MemSys ’25. Association for Computing Machinery, 2026, pp. 262–270. ISBN: 9798400720024. DOI: 10.1145/3767110.3767119.

- [43] J. Kiszka. *Reworking the Inter-VM Shared Memory Device*. 2019. URL: https://static.sched.com/hosted_files/kvmforum2019/4b/KVM-Forum19_ivshmem2.pdf (visited on 08/10/2026).
- [44] K. Kobori, C. Lee, and T. Hirotsu. "A Portable Time Synchronization Method using eBPF." In: *2022 Tenth International Symposium on Computing and Networking Workshops (CANDARW)*. 2022, pp. 20–25. DOI: 10.1109/CANDARW57323.2022.00045.
- [45] J. W. Konka, C. M. Arthur, F. J. Garcia, and R. C. Atkinson. "Traffic generation of IEC 61850 sampled values." In: *2011 IEEE First International Workshop on Smart Grid Modeling and Simulation (SGMS)*. 2011, pp. 43–48. DOI: 10.1109/SGMS.2011.6089025.
- [46] L. Lai, G. Ara, T. Cucinotta, K. Kondepu, and L. Valcarengi. "Ultra-low Latency NFV Services Using DPDK." In: *2021 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. 2021, pp. 8–14. DOI: 10.1109/NFV-SDN53031.2021.9665131.
- [47] C. Li, C. Ding, and K. Shen. "Quantifying the cost of context switch." In: *Proceedings of the 2007 Workshop on Experimental Computer Science*. ExpCS '07. San Diego, California: Association for Computing Machinery, 2007, 2–es. ISBN: 9781595937513. DOI: 10.1145/1281700.1281702.
- [48] *Libxdp function xsk_socket__create_shared*. eBPF.io. July 28, 2024. URL: https://docs.ebpf.io/ebpf-library/libxdp/functions/xsk_socket__create_shared/ (visited on 08/05/2026).
- [49] S. Lindner, A. Shahraki, and D. Schulz. "Performance Evaluation of Linux-Based Parallel Redundancy Protocol (PRP) for Redundant Industrial Networks." In: *IEEE Access* 14 (2026), pp. 9952–9960. DOI: 10.1109/ACCESS.2026.3655094.
- [50] *Linux eBPF Documentation*. eBPF.io. July 14, 2026. URL: <https://docs.ebpf.io/linux/> (visited on 08/12/2026).
- [51] *Linux Socket Filtering aka Berkeley Packet Filter (BPF)*. The Linux Kernel Documentation. URL: <https://docs.kernel.org/networking/filter.html> (visited on 08/15/2026).
- [52] J. C. Lozano, K. Koneru, N. Ortiz, and A. A. Cardenas. "Digital Substations and IEC 61850: A Primer." In: *IEEE Communications Magazine* 61.6 (2023), pp. 28–34. DOI: 10.1109/MCOM.001.2200568.
- [53] R. Mackiewicz. "Overview of IEC 61850 and Benefits." In: *2006 IEEE PES Power Systems Conference and Exposition*. 2006, pp. 623–630. DOI: 10.1109/PSCE.2006.296392.

- [54] F. Maurer. *[PATCH net-next v4 0/8] hsr: Implement more robust duplicate discard algorithm*. Feb. 5, 2026. URL: <https://lwn.net/ml/all/cover.1770299429.git.fmaurer@redhat.com/> (visited on 08/12/2026).
- [55] F. Maurer. *[PATCH net-next v4 4/8] hsr: Implement more robust duplicate discard for PRP*. lore.kernel.org public-inbox archive. Feb. 5, 2026. URL: <https://lore.kernel.org/all/8ce15a996099df2df5b700969a39e7df400e8dbb.1770299429.git.fmaurer@redhat.com/> (visited on 08/12/2026).
- [56] *memfd_create(2)* — *Linux manual page*. man7.org. Feb. 8, 2026. URL: https://man7.org/linux/man-pages/man2/memfd_create.2.html (visited on 08/18/2026).
- [57] J. L. Messenger. “Time-Sensitive Networking: An Introduction.” In: *IEEE Communications Standards Magazine* 2.2 (2018), pp. 29–33. DOI: 10.1109/MCOMSTD.2018.1700047.
- [58] *Network bridge*. ArchWiki. URL: https://wiki.archlinux.org/title/Network_bridge (visited on 08/04/2026).
- [59] *PCI-SIG Single Root I/O Virtualization (SR-IOV) Support in Intel® Virtualization Technology for Connectivity*. Intel. URL: <https://www.intel.com/content/dam/doc/white-paper/pci-sig-single-root-io-virtualization-support-in-virtualization-technology-for-connectivity-paper.pdf> (visited on 08/06/2026).
- [60] B. Pfaff, J. Pettit, T. Koponen, E. Jackson, A. Zhou, J. Rajahalme, J. Gross, A. Wang, J. Stringer, P. Shelar, K. Amidon, and M. Casado. “The Design and Implementation of Open vSwitch.” In: *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. Oakland, CA: USENIX Association, May 2015, pp. 117–130. ISBN: 978-1-931971-218.
- [61] *Program type BPF_PROG_TYPE_SCHED_ACT*. ebpf.io. May 8, 2026. URL: https://docs.ebpf.io/linux/program-type/BPF_PROG_TYPE_SCHED_ACT/ (visited on 08/12/2026).
- [62] *Program type BPF_PROG_TYPE_XDP*. ebpf.io. June 18, 2026. URL: https://docs.ebpf.io/linux/program-type/BPF_PROG_TYPE_XDP/ (visited on 08/12/2026).
- [63] *Programmer’s Guide - 1. Environment Abstraction Layer (EAL) Library*. DPDK Project. URL: https://doc.dpdk.org/guides/prog_guide/env_abstraction_layer.html (visited on 08/10/2026).
- [64] *Programmer’s Guide - 15. Poll Mode Driver*. DPDK Project. URL: https://doc.dpdk.org/guides-24.03/prog_guide/poll_mode_drv.html (visited on 08/03/2026).

- [65] *Programmer's Guide - 2. Overview*. DPDK Project. URL: https://doc.dpdk.org/guides/prog_guide/overview.html (visited on 08/07/2026).
- [66] *Programmer's Guide - 3. Ethernet Device Library*. DPDK Project. URL: https://doc.dpdk.org/guides/prog_guide/ethdev/flow_offload.html (visited on 08/10/2026).
- [67] *Programmer's Guide - 46. Vhost Library*. DPDK Project. URL: https://doc.dpdk.org/guides-19.11/prog_guide/vhost_lib.html (visited on 08/18/2026).
- [68] F. Reghenzani, G. Massari, and W. Fornaciari. "The Real-Time Linux Kernel: A Survey on PREEMPT_RT." In: *ACM Comput. Surv.* 52.1 (Feb. 2019). ISSN: 0360-0300. DOI: 10.1145/3297714.
- [69] *rte_ethdev.h File Reference*. DPDK Project. URL: https://doc.dpdk.org/api/rte_ethdev_8h.html (visited on 08/10/2026).
- [70] *RUGGEDCOM Compact Switches*. Siemens. 2018. URL: https://cache.industry.siemens.com/dl/files/347/109766347/att_980973/v1/SIE_compactlayer_2_ethernet_switches.pdf (visited on 08/01/2026).
- [71] R. Salazar, T. Godfrey, N. Finn, C. Powell, B. Rolfe, and M. Seewald. "Utility Applications of Time Sensitive Networking White Paper." In: *Utility Applications of Time Sensitive Networking White Paper* (2019), pp. 1–19.
- [72] *Scaling in the Linux Networking Stack*. The Linux Kernel Documentation. URL: <https://docs.kernel.org/networking/scaling.html> (visited on 08/03/2026).
- [73] S. Schönborn, R. Birke, D. Kozhaya, and T. Sivanthi. "Real-time performance of virtualised protection and control software." In: *27th International Conference on Electricity Distribution (CIRED 2023)*. Vol. 2023. 2023, pp. 1817–1821. DOI: 10.1049/icp.2023.1028.
- [74] M. Seewald. *Communication Networks and Systems for Power Utility Automation*. Cisco. May 24, 2021. URL: <https://blogs.cisco.com/networking/communication-networks-and-systems-for-power-utility-automation> (visited on 08/01/2026).
- [75] Siemens. *SIPROTEC V*. URL: <https://www.siemens.com/de-de/products/siprotec/siprotec-v/> (visited on 07/27/2026).
- [76] S. A. Siewior. *How realtime kernels differ*. The Linux Kernel Documentation. URL: <https://docs.kernel.org/core-api/real-time/differences.html> (visited on 08/04/2026).
- [77] *struct sk_buff*. The Linux Kernel Documentation. URL: <https://docs.kernel.org/networking/skbuff.html> (visited on 08/14/2026).

- [78] *VirtIO Devices*. QEMU Project. 2026. URL: <https://www.qemu.org/docs/master/system/devices/virtio/index.html> (visited on 08/05/2026).
- [79] *Virtualization Host Configuration and Guest Installation Guide - Chapter 13. SR-IOV*. Red Hat. URL: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/virtualization_host_configuration_and_guest_installation_guide/chap-virtualization_host_configuration_and_guest_installation_guide-sr_iov (visited on 08/06/2026).
- [80] C. Xue, T. Zhang, and S. Han. "Towards Cost-Effective Real-Time High-Throughput End Station Design for Time-Sensitive Networking (TSN)." In: *Proceedings of the 61st ACM/IEEE Design Automation Conference*. DAC '24. San Francisco, CA, USA: Association for Computing Machinery, 2024. ISBN: 9798400706011. DOI: 10.1145/3649329.3658260.
- [81] T. Zhang, G. Wang, C. Xue, J. Wang, M. Nixon, and S. Han. "Time-Sensitive Networking (TSN) for Industrial Automation: Current Advances and Future Directions." In: *ACM Comput. Surv.* 57.2 (Oct. 2024). ISSN: 0360-0300. DOI: 10.1145/3695248.